Wave: Adaptive Procedural Audio for Sleep and Focus

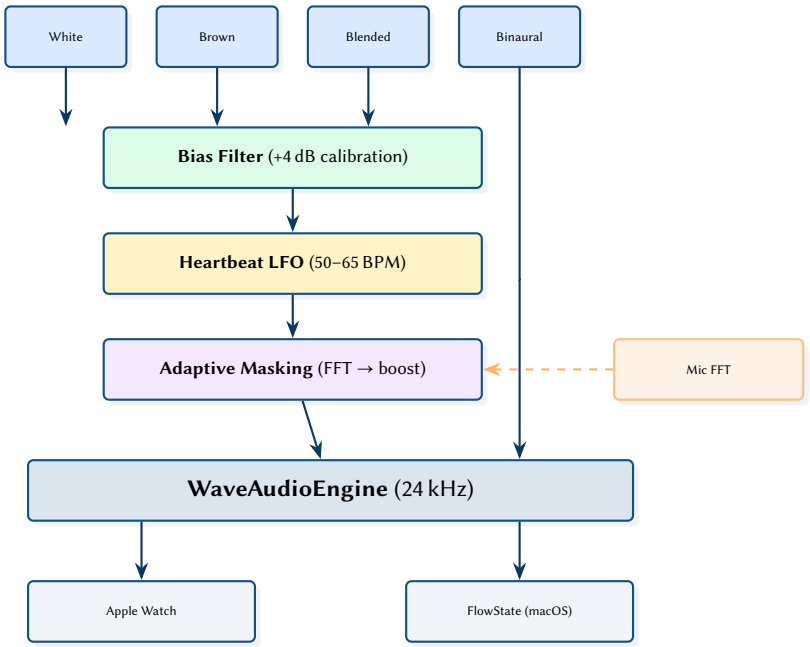
Real-Time Synthesis, Adaptive Masking, and Wrapper Composition on watchOS and macOS

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 16, 2026



Abstract. Wave is an adaptive ambient audio ecosystem spanning Apple Watch (StillState—sleep) and macOS (FlowState—focus). All audio is generated procedurally via `AVAudioSourceNode` at 24 kHz, producing white noise, brown noise (first-order IIR), time-varying blends, and binaural beats. A **wrapper composition** architecture stacks processing layers—frequency calibration bias, heartbeat-synchronised amplitude modulation, and adaptive environmental masking—onto base generators without modifying their code. The adaptive masking subsystem uses a **dual audio engine** design: a dedicated microphone engine performs Hann-windowed RMS analysis at 2 Hz, feeding an EMA-smoothed control law with tanh saturation that adjusts playback volume up to 1.5× via 30-second exponential ramps. Real-time thread safety is achieved through `OSAllocatedUnfairLock` with full value-copy semantics, ensuring lock hold times remain below 1 μs. The system targets <12% battery drain per hour (playback only) and <15% with all adaptive features enabled, supporting all-night 8-hour sleep sessions on Apple Watch Series 4+.

Contents

Part I

Audio Synthesis

1 Introduction and Motivation

The use of ambient noise for sleep has a well-established evidence base. Stanchina et al. [1] demonstrated that white noise reduced the difference between baseline and peak noise events in an ICU setting, significantly reducing arousal frequency. Messineo et al. [2] conducted a systematic review confirming that broadband noise decreases sleep onset latency in adults exposed to environmental noise. Despite this evidence, most consumer sleep-sound applications play pre-recorded audio loops, introducing two problems:

1. **Loop seam detection.** Light sleepers can perceive the repeat point of looped audio, triggering micro-arousals. Perceptual studies on auditory continuity [3] show that sudden spectral discontinuities at loop boundaries activate the auditory stream segregation mechanism, breaking the illusion of a continuous sound source.
2. **Storage and battery cost.** High-quality uncompressed audio files consume significant storage on a constrained device like Apple Watch (32 GB shared with watchOS). Decoding compressed formats (e.g. AAC, Opus) imposes additional CPU overhead and corresponding battery drain.

Wave addresses these constraints by synthesising all audio procedurally at render time. The app runs entirely on Apple Watch, streaming audio directly to Bluetooth headphones from the wrist. A parallel macOS menu bar app (FlowState) provides the same synthesis engine for focus work.

Insight: *Procedural synthesis eliminates both problems: infinite non-repeating output with zero storage and no decoding overhead. The cost is paid in CPU cycles for real-time generation, but the algorithms involved are remarkably cheap.*

: Procedural Generation Only

All audio is generated using mathematical functions at render time. No audio files are bundled with the app. This eliminates storage overhead, removes loop boundaries, and provides infinite non-repeating output.

: Battery-First Design

All adaptive features default to **off**. The base noise generators are the lowest-cost path. Features are additive layers the user opts into, never mandatory background processes.

1.1 The Generator Protocol

All noise sources in Wave implement a common protocol:

```
public protocol StereoNoiseGenerator: Sendable {
    func generateStereoSample() -> (left: Float, right: Float)
}
```

The `Sendable` conformance is required because generators cross thread boundaries—created on the main thread, consumed in the audio render callback. This protocol forms the basis for the wrapper composition architecture described in section 6.

2 White Noise

White noise is defined as a signal with flat power spectral density across all frequencies [4]. In the discrete-time domain, this is approximated by independent, identically distributed (i.i.d.) random samples drawn from a uniform distribution $U(-1, 1)$.

2.1 Moving Average Smoothing

Rather than emitting raw random samples, Wave applies a short Finite Impulse Response (FIR) moving average filter to reduce the harshness of high-frequency energy. The filter computes a running mean over the most recent N samples using a circular ring buffer:

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \tag{1}$$

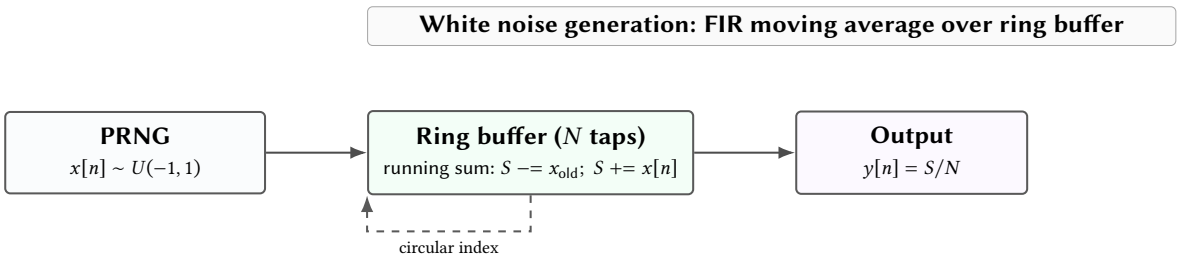


Figure 1. White noise via FIR moving average. The running sum avoids recomputing the full average each sample—cost is $O(1)$ per sample regardless of tap count.

The tap count N is configurable by performance mode:

Table 1. White noise filter taps by performance mode.

Mode	N (taps)	Effect
Quality	24	Smoothest; gentle HF rolloff
Balanced	18	Default
Battery	12	Crispest; lowest CPU cost

Stereo is achieved by running two fully independent generators, producing uncorrelated left and right channels.

Insight: *A moving average of length N has a frequency response of $H(f) = \frac{\sin(\pi f N)}{\sin(\pi f) \cdot N}$, producing nulls at $f = k/N$. At $N = 18$ and $f_s = 24$ kHz, the first null falls at 1333 Hz—well above the sleep-relevant band but providing a gentle rolloff that softens the perceived "hissiness" of raw white noise.*

Design Decision 1: Performance Mode

The three-level performance mode (Quality / Balanced / Battery) controls *only* the FIR filter tap count. Sample rate (24 kHz), buffer size (256 samples), and feature availability are invariant across modes. The blended generator caps its white noise component at $\min(N, 12)$ taps regardless of mode, since the blend's spectral character is dominated by the brown noise component. Brown noise and binaural beat generators are entirely unaffected by the performance setting.

3 Brown Noise

Brown noise (also called red noise or Brownian noise) exhibits a power spectral density proportional to $1/f^2$, giving it a 6 dB/octave rolloff [5]. The name derives from Robert Brown's observation of Brownian motion, not from a colour.

3.1 The One-Pole IIR Filter

Brown noise is generated by passing white noise through a first-order Infinite Impulse Response (IIR) low-pass filter. The transfer function of a one-pole filter is [6]:

$$H(z) = \frac{a_0}{1 - b_1 z^{-1}} = \frac{1 - x}{1 - x \cdot z^{-1}} \tag{2}$$

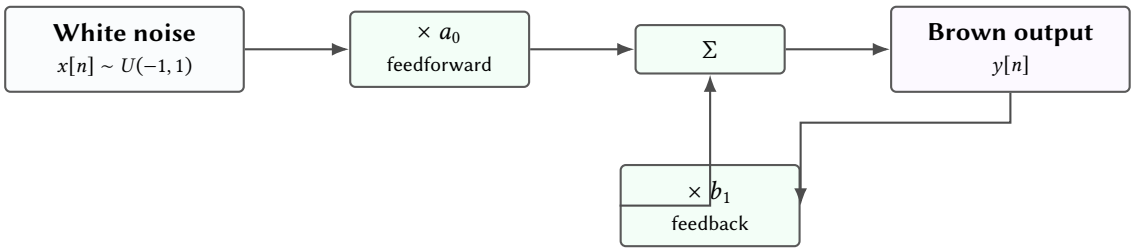
where the pole position x is derived from the desired cutoff frequency:

$$x = e^{-2\pi f_c / f_s} \tag{3}$$

The difference equation for the output $y[n]$ given input $x[n]$ is:

$$y[n] = a_0 \cdot x[n] + b_1 \cdot y[n-1] \tag{4}$$

One-pole IIR: shaping white noise into brown with a single state variable



Computational cost: one multiply-add per sample plus one state variable. This is the cheapest possible spectral shaping—essential for real-time synthesis on Apple Watch's constrained CPU budget.

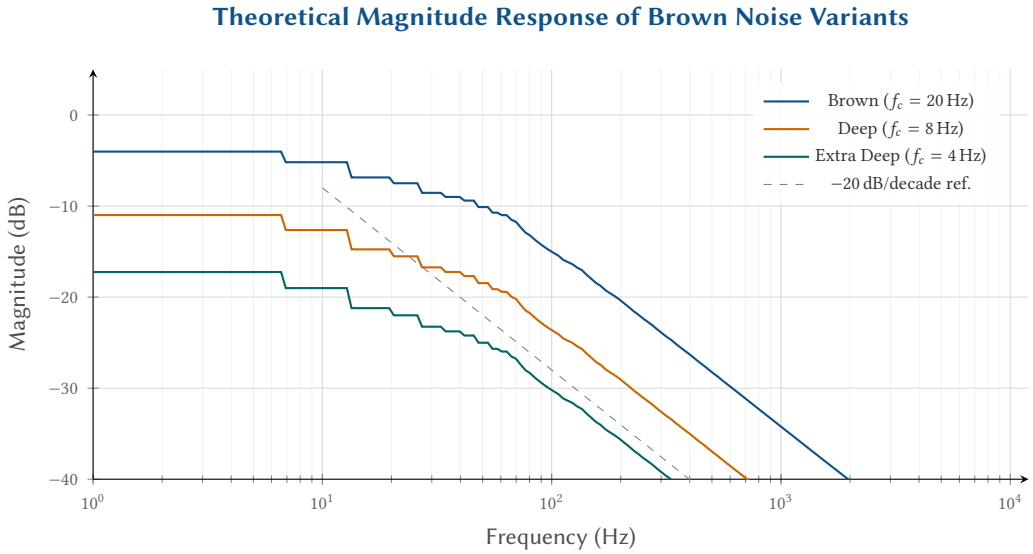
Figure 2. Signal flow graph of the one-pole IIR low-pass filter used for brown noise generation. The feedback coefficient b_1 controls the spectral slope.

3.2 Spectral Variants

Wave provides three brown noise variants, each with a different pole position. Moving the pole closer to 1.0 concentrates more energy in the sub-bass:

Table 2. Brown noise variants and their pole positions at $f_s = 24\,\text{kHz}$.

Variant	f_c (Hz)	b_1	Character
Brown	20	0.99476	Warm, full-spectrum rumble
Deep Brown	8	0.99791	Heavy low-end emphasis
Extra Deep	4	0.99895	Sub-bass dominant, almost felt



Insight: *With $b_1 = 0.99895$ (Extra Deep), over 99.8% of the output energy comes from feedback. The generator is very nearly a random walk, producing the deep rumble that many users find most effective for sleep masking.*

Figure 3. Magnitude response of the three brown noise variants. All exhibit the characteristic $1/f^2$ (−20 dB/decade) rolloff. Lower cutoff frequencies shift the "knee" further into the sub-bass.

4 Blended Noise

The blended generator produces a time-varying mix of white and brown noise. This exploits the psychoacoustic principle of *habituation reduction*—the auditory system adapts to constant stimuli over time, reducing their masking effectiveness [3]. Gentle spectral variation prevents the listener's auditory cortex from fully habituating.

4.1 LFO-Modulated Crossfade

The blend ratio is controlled by a sinusoidal Low-Frequency Oscillator (LFO):

$$m(t) = 0.5 + 0.1 \cdot \sin(2\pi f_{\text{LFO}} \cdot t) \tag{5}$$

where $f_{\text{LFO}} = 0.05\,\text{Hz}$ (20-second cycle). The output is:

$$y[n] = m(t) \cdot x_w[n] + (1 - m(t)) \cdot x_b[n] \tag{6}$$

The blend ratio swings between 0.4 (60% brown, "darker") and 0.6 (60% white, "brighter") over the 20-second cycle.

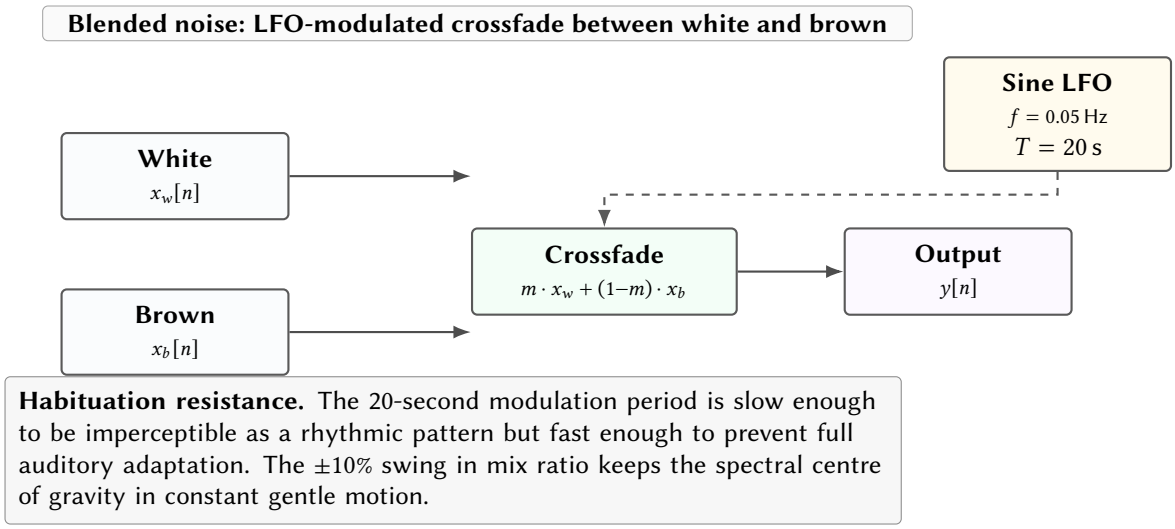


Figure 4. Blended noise generator. A sinusoidal LFO modulates the crossfade coefficient between white and brown noise at 0.05 Hz.

5 Binaural Beats

Binaural beats are an auditory illusion first described by Dove [7] and popularised by Oster [8]. When two tones of slightly different frequency are presented to separate ears via headphones, the listener perceives a rhythmic "beating" at the difference frequency. This perceptual phenomenon arises from neural processing in the superior olivary complex, which computes interaural phase differences for sound localisation [9].

Insight:
The 20-second LFO period was chosen empirically: shorter periods (< 5 s) create a perceptible "pulsing" quality; longer periods (> 60 s) allow habituation to set in before the spectrum shifts. The 20 s sweet spot provides variation without rhythmic awareness.

5.1 Brainwave Entrainment

The hypothesised mechanism of action is *frequency following response* (FFR): the brain's dominant Electroencephalography (EEG) frequency tends to synchronise with periodic auditory stimuli [10]. Beat frequencies in different bands target different brain states:

Table 3. Binaural beat presets and their target brainwave bands.

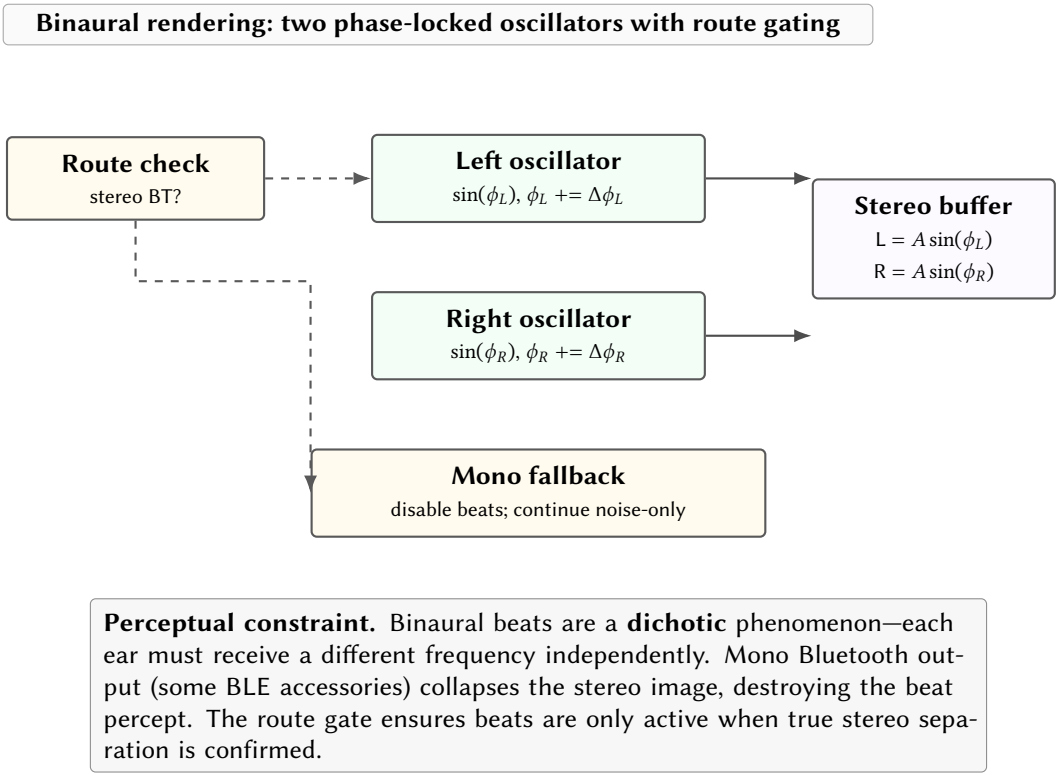
Preset	Band	Beat (Hz)	Carrier (Hz)	Target State
Deep Sleep	Delta	2	200	Stage 3/4 NREM sleep
Sleep	Delta	4	200	Sleep onset
Meditation	Theta	6	250	Meditative relaxation
Relaxation	Alpha	8	250	Calm wakefulness
Focus	Beta	15	300	Sustained attention

Users can also specify custom carrier (50–1000 Hz) and beat (0.5–40 Hz) frequencies. Carrier frequencies are clamped to ensure both tones remain above 20 Hz and below Nyquist.

5.2 Phase Accumulator Implementation

The generator uses two independent phase accumulators in double precision to avoid frequency drift over long playback sessions:

$$f_L = f_{\text{carrier}} - f_{\text{beat}}/2 \tag{7}$$
$$f_R = f_{\text{carrier}} + f_{\text{beat}}/2 \tag{8}$$
$$\Delta\phi_L = 2\pi f_L/f_s \qquad \Delta\phi_R = 2\pi f_R/f_s \tag{9}$$



Perceptual constraint. Binaural beats are a **dichotic** phenomenon—each ear must receive a different frequency independently. Mono Bluetooth output (some BLE accessories) collapses the stereo image, destroying the beat percept. The route gate ensures beats are only active when true stereo separation is confirmed.

Figure 5. Binaural beat generation with route gating. Beats are suppressed on mono output routes to prevent imperceptible (and therefore useless) playback.

Phase accumulators use `Double` precision internally, casting to `Float` only at output. At 24 kHz and a worst-case carrier of 1000 Hz, the phase increment is ≈ 0.262 rad/sample. After 8 hours of continuous playback ($\approx 6.9 \times 10^8$ samples), the accumulated phase error with `Float32` would reach ≈ 0.07 radians ($\approx 4^\circ$), sufficient to create audible pitch drift. `Double` precision eliminates this entirely.

5.3 Composite Mixing

When users enable both binaural beats and a noise bed, a `CompositeStereoGenerator` mixes the two signals at fixed gains:

$$y[n] = 0.6 \cdot s_{\text{beats}}[n] + 0.3 \cdot s_{\text{noise}}[n] \tag{10}$$

Output is hard-clamped to $[-1, 1]$. In practice, clipping never occurs: the binaural amplitude is 0.2 (-14 dBFS) and the noise floor is bounded by the one-pole filter.

6 Wrapper Composition Architecture

Rather than building monolithic generator classes with conditional logic for each feature, Wave uses a **wrapper composition** (decorator) pattern [11]. Each processing layer implements `StereoNoiseGenerator` and delegates to an inner generator, applying its own transformation to each sample pair.

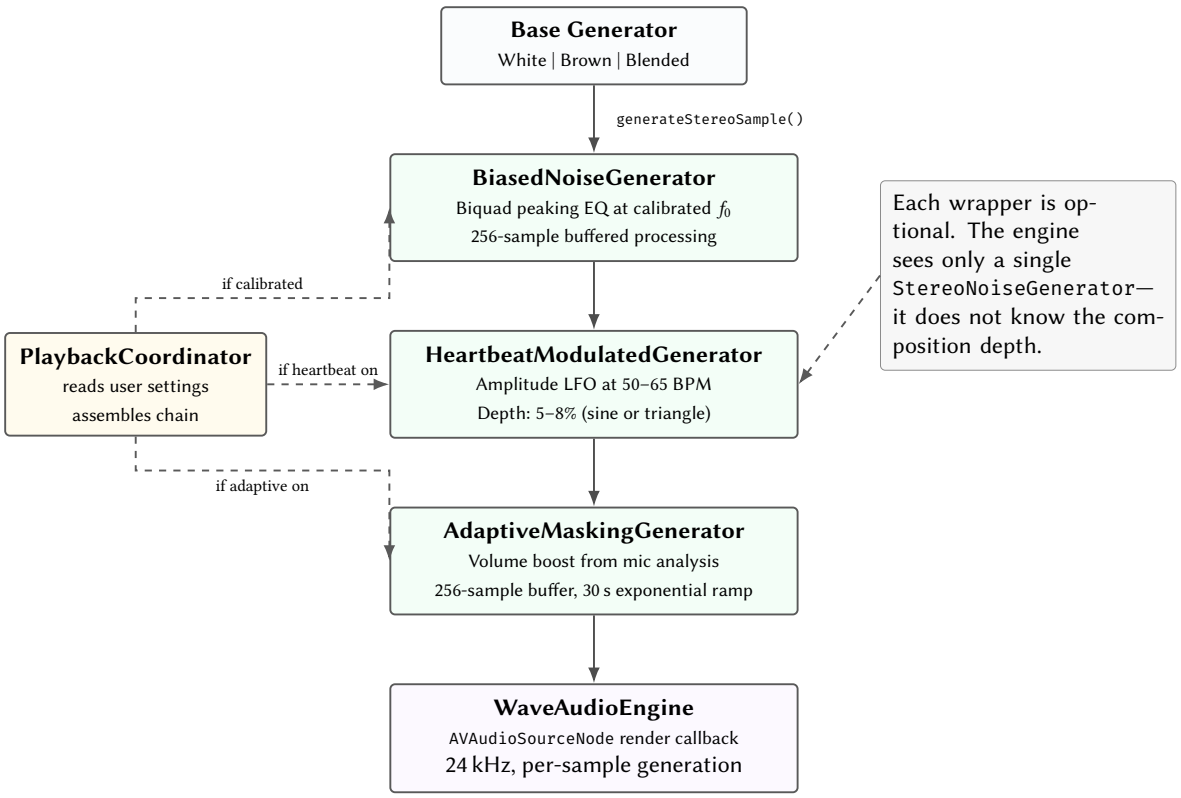


Figure 6. Wrapper composition chain. Processing layers are assembled dynamically based on user preferences. Each layer delegates to its inner generator, applying one transformation. The engine is agnostic to the chain structure.

: Wrapper Composition

Processing layers are added by wrapping, not by modifying base generators. Each generator knows only about its own transformation. The composition chain is assembled dynamically at playback start based on the current user preferences.

The assembly order is fixed:

1. **Base generator** (white, brown, deep brown, extra deep brown, or blend)
2. **BiasedNoiseGenerator** — if calibration is complete and enabled
3. **HeartbeatModulatedGenerator** — if heartbeat sync is enabled
4. **AdaptiveMaskingGenerator** — if adaptive masking is enabled

When any parameter changes mid-playback, a new chain is built and hot-swapped into the running engine without stopping audio (see section 11.2).

Insight: *Adding a new processing layer requires zero changes to existing generators—just a new wrapper class implementing the protocol. The environment synthesis feature (Phase 27) will use exactly this mechanism: a new wrapper that mixes procedural environmental sounds into the noise bed.*

Part II

Adaptive Intelligence

7 Frequency Calibration

Individual frequency sensitivity varies significantly due to ear canal resonance, outer ear geometry (pinna effect), and age-related hearing loss (presbycusis) [12]. The calibration subsystem identifies each user's preferred frequency via an interactive sweep, then applies a persistent spectral bias to all subsequent playback.

7.1 Calibration Wizard

The calibration subsystem presents a three-step interactive flow (Intro → Testing → Results) driven by a dedicated `CalibrationEngine`. The engine operates its own `AVAudioEngine` instance—separate from the playback engine—so that calibration tones never interfere with an active noise session.

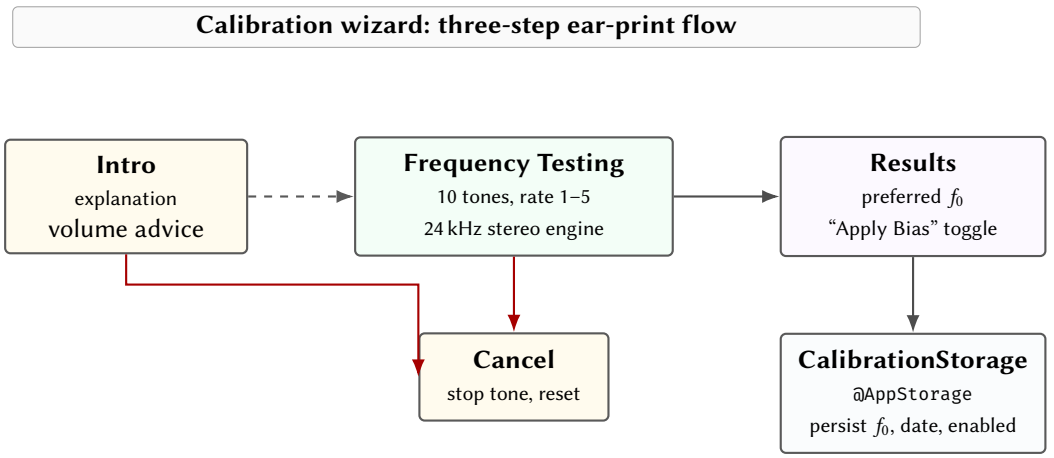


Figure 7. Calibration wizard flow. The user completes a 10-frequency sweep, rating each tone for subjective relaxation. Results are persisted and optionally applied as a biquad bias.

The `CalibrationToneGenerator` produces pure sine tones at 24 kHz sample rate with an amplitude of ≈ -18 dBFS (0.12 linear). Ten frequencies are presented in a logarithmic sweep:

Table 4. Calibration sweep frequencies (logarithmic spacing).

1	2	3	4	5	6	7	8	9	10
20	40	80	160	320	640	1250	2500	5000	8000

At each step the user rates the tone from 1 (unpleasant) to 5 (most relaxing). The preferred frequency is selected as the highest-rated tone. If all ratings are identical (no preference), the system defaults to $f_0 = 400$ Hz—a frequency in the upper-bass/lower-midrange region where most people find noise subjectively warm.

Results are persisted to four `@AppStorage` keys: `calibrationPreferredHz`, `calibrationCompleted`, `calibrationBiasEnabled`, and `calibrationDate`. The bias toggle can be flipped post-calibration in Settings, triggering a live hot-swap of the generator chain.

Insight: *The calibration engine requires at least three rated frequencies and at least two distinct ratings before accepting a preference. This prevents a user who taps “3” on every tone from receiving a spurious bias at 20 Hz.*

7.2 The Peaking EQ Biquad Filter

The calibration result is applied via a biquad peaking EQ filter [13]. Unlike shelving filters (which boost everything above or below a frequency) or bandpass filters (which attenuate everything outside the passband), a peaking EQ applies a localised gain boost centred at a single frequency, leaving the rest of the spectrum untouched.

The transfer function of a second-order biquad is:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}} \tag{11}$$

For a peaking EQ, the coefficients are derived from the Audio EQ Cookbook [13]:

$$\omega = 2\pi f_0 / f_s \tag{12}$$
$$\alpha = \sin(\omega)/(2Q) \tag{13}$$
$$A = 10^{G_{\text{dB}}/40} \tag{14}$$

$$b_0 = 1 + \alpha A \qquad a_0 = 1 + \alpha/A \tag{15}$$
$$b_1 = -2\cos\omega \qquad a_1 = -2\cos\omega \tag{16}$$
$$b_2 = 1 - \alpha A \qquad a_2 = 1 - \alpha/A \tag{17}$$

All coefficients are normalised by a_0 before use.

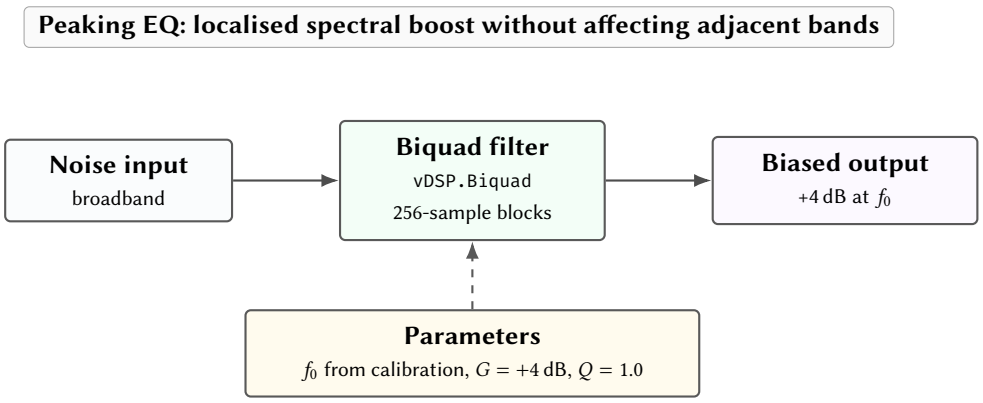
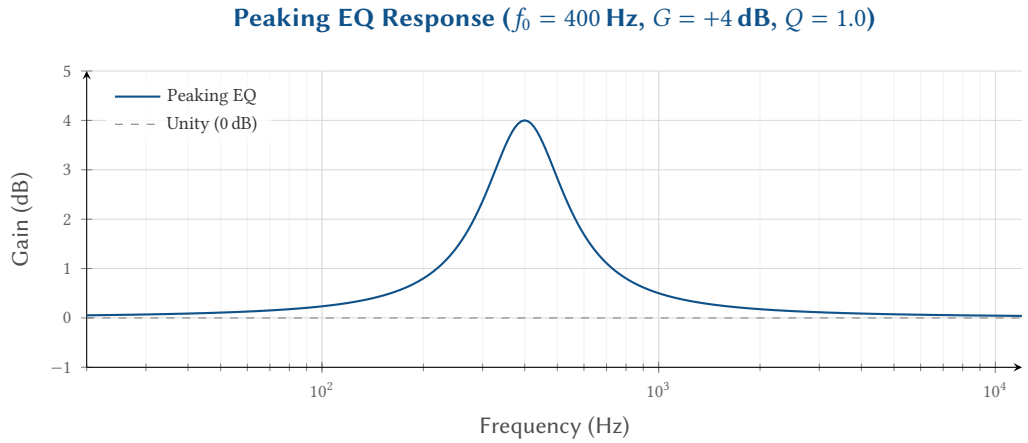


Figure 8. Frequency bias via peaking EQ biquad filter. The user's calibrated frequency receives a +4 dB boost; all other frequencies pass through unaffected.

Default parameters: $f_0 = 400$ Hz (user-calibrated), $G = +4$ dB, $Q = 1.0$. The filter is implemented via `vDSP.Biquad` (Accelerate framework [14]) processing 256-sample blocks in double precision internally.



Insight: *A 4 dB boost at the user's preferred frequency is subtle enough to avoid conscious perception as "coloured" noise but sufficient to increase subjective comfort. The Q of 1.0 gives a bandwidth of approximately one octave, wide enough to feel natural rather than resonant.*

Figure 9. Approximate magnitude response of the peaking EQ with default parameters. The boost is localised around 400 Hz with approximately one-octave bandwidth ($Q = 1.0$).

8 Heartbeat Synchronisation

The heartbeat modulation layer applies a gentle rhythmic amplitude envelope to the noise, synchronised to a target heart rate. The principle is *auditory-cardiac coupling*: rhythmic auditory stimuli at near-resting heart rate can entrain autonomic nervous system activity, promoting parasympathetic dominance and relaxation [15].

8.1 LFO Design

The modulation source is a phase-accumulator LFO running at the audio sample rate (24 kHz):

$$\Delta\phi = \frac{2\pi \cdot (\text{BPM}/60)}{f_s} \tag{18}$$

Two waveform shapes are available, selected automatically based on target BPM:

- **Sine** (BPM ≤ 55): $e[n] = (\sin(\phi) + 1)/2$. Smooth, no abrupt transitions—best for deep sleep.
- **Triangle** (BPM > 55): linear ramp up in the first half-cycle, linear ramp down in the second. Slightly sharper pulse—more perceptible, suitable for grounding during relaxation.

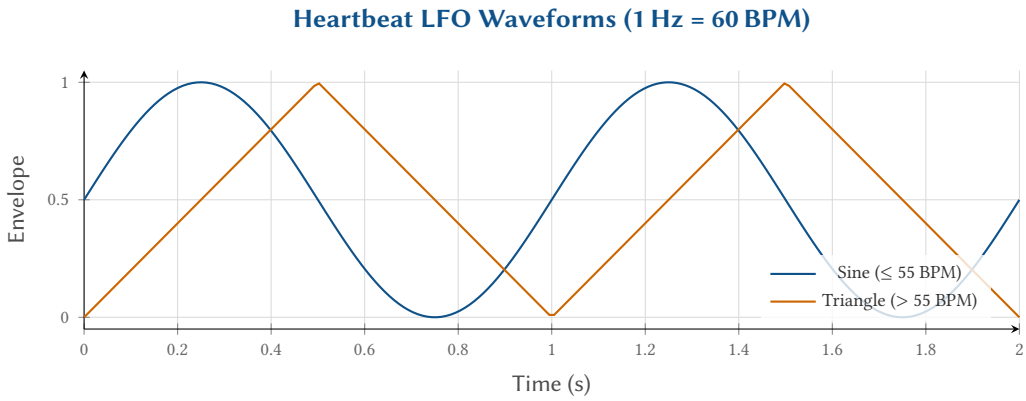


Figure 10. Heartbeat LFO waveforms at 60 BPM (1 Hz). The sine shape provides the smoothest modulation for deep sleep presets; the triangle shape provides a slightly sharper pulse for relaxation.

8.2 Modulation Depth

The envelope maps to a gain multiplier:

$$g[n] = 1.0 - d \cdot (1.0 - e[n]) \tag{19}$$

where $d \in [0.05, 0.08]$ (default 0.065) is the modulation depth and $e[n] \in [0, 1]$ is the normalised LFO output.

Table 5. Heartbeat modulation presets.

Preset	BPM	Depth	Shape	Gain Range
Deep Rest	50	6.5%	Sine	0.935–1.0
Resting HR	55	6.5%	Sine	0.935–1.0
Relaxed	60	6.5%	Triangle	0.935–1.0
Light Activity	65	6.5%	Triangle	0.935–1.0

At default depth ($d = 0.065$), the amplitude varies between 0.935 (trough) and 1.0 (peak)—a 6.5% swing. This is below the just-noticeable difference for amplitude modulation in the presence of noise [16], making it imperceptible as a conscious rhythm while still providing a periodic autonomic cue.

> **Binaural Carrier Protection**
>
> Heartbeat modulation must **never** be applied to binaural beat carrier oscillators. Amplitude-modulating the carrier introduces sidebands at $f_{\text{carrier}} \pm f_{\text{beat}}$, destroying the clean frequency separation that the binaural effect requires. The composition chain enforces this: when binaural beats are enabled, the function returns at the composite generator without entering the heartbeat/adaptive wrapping path.

BPM changes are applied by updating the phase increment without resetting the current phase, ensuring glitch-free transitions.

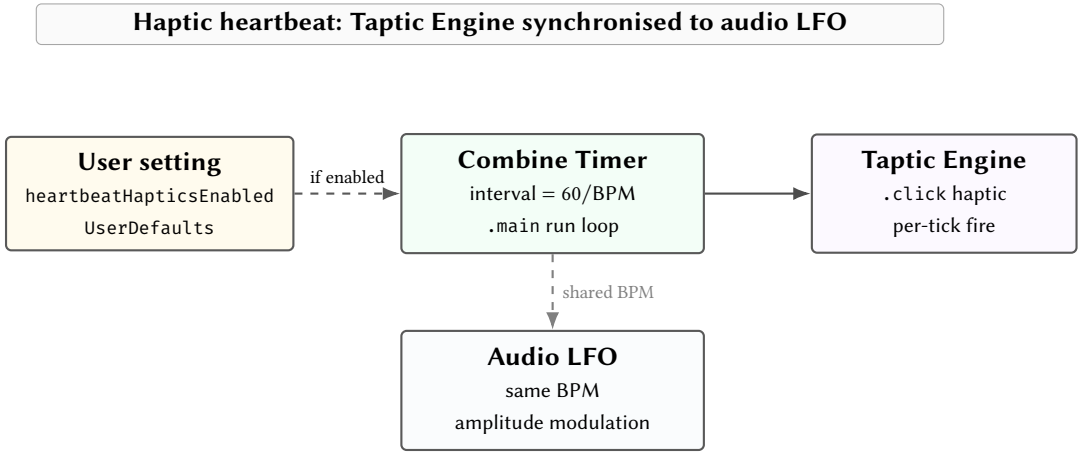
8.3 Tactile Reinforcement: Heartbeat Haptic Engine

In addition to auditory modulation, the Apple Watch can deliver synchronised tactile pulses to the user's wrist via the Taptic Engine. Azevedo et al. [17] demonstrated that rhythmic haptic stimulation simulating a slow heartbeat produces a significant calming effect during stress, even when participants are unaware the stimulus is externally generated. Costa et al. [18] showed that false cardiac feedback delivered via wearable haptics can modulate self-reported emotional arousal. Wave exploits these findings by coupling audio and haptic modalities at the same BPM.

The `HeartbeatHapticEngine` runs a Combine `Timer.publish` at an interval derived from the BPM setting:

$$T_{\text{haptic}} = \frac{60}{\text{BPM}} \text{ seconds} \tag{20}$$

On each tick, the engine calls `WKInterfaceDevice.play(.click)`, producing a subtle wrist tap. At 60 BPM the interval is exactly 1 second; at 50 BPM it stretches to 1.2 seconds.



Dual-modality entrainment. The audio LFO and haptic timer share the same BPM source, delivering correlated auditory and somatosensory cues. This multimodal approach may strengthen autonomic entrainment compared to either modality alone.

Figure 11. Heartbeat haptic engine. A Combine timer fires at the same BPM as the audio LFO, delivering synchronised wrist taps via the Taptic Engine.

The haptic engine supports live toggling via `refreshEnabledState()`: if the user disables haptics mid-session, the timer is cancelled without disturbing the audio LFO; re-enabling restarts the timer at the current BPM. BPM updates while running transparently tear down and recreate the timer at the new interval.

: Haptic–Audio Independence

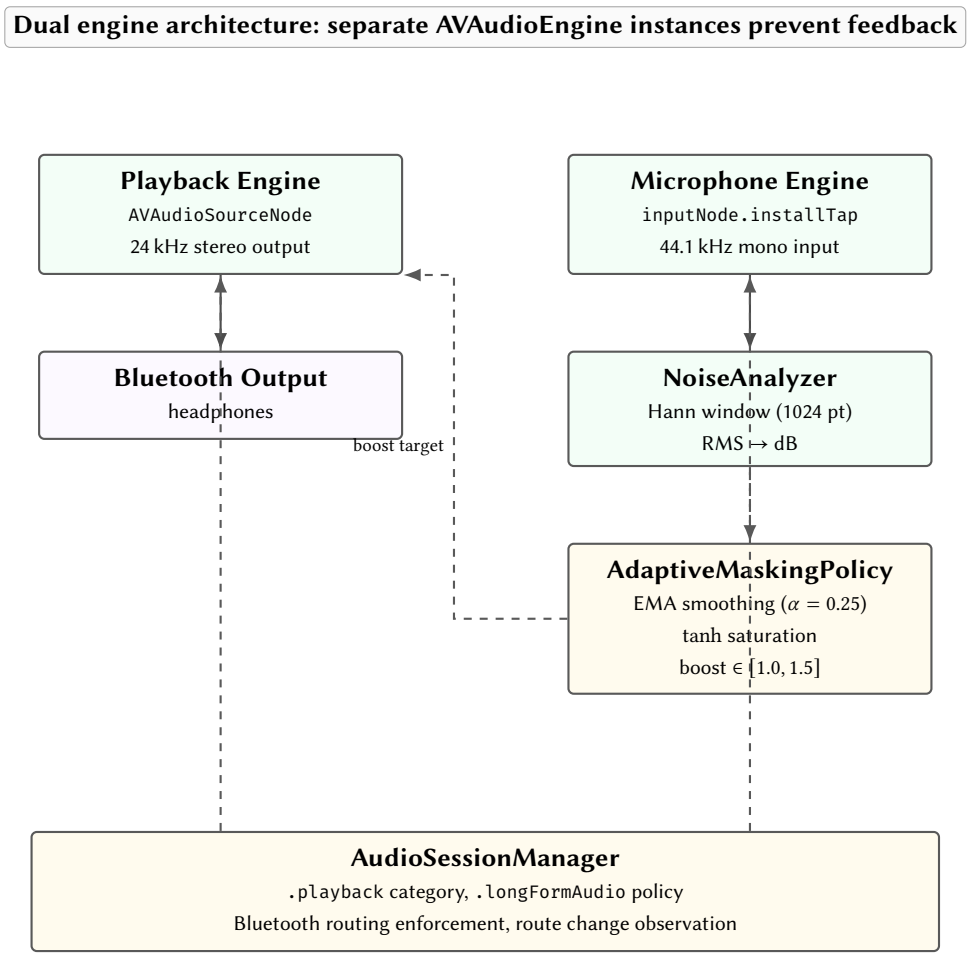
The haptic timer and audio LFO share a BPM source but are otherwise fully independent. Disabling haptics has zero effect on audio modulation, and vice versa. This allows the user to choose auditory-only, tactile-only, or combined entrainment.

9 Adaptive Environmental Masking

The adaptive masking subsystem monitors ambient noise via the watch microphone and automatically adjusts playback volume to maintain effective masking. The theoretical basis is the *masking threshold* [12]: a target sound (snoring, traffic) is masked when the noise floor exceeds the target's audibility threshold at each critical band. When ambient noise rises, the playback volume must rise proportionally to maintain the masking margin.

9.1 Dual Audio Engine Architecture

A fundamental constraint of real-time audio monitoring is preventing feedback: the microphone must not pick up the playback output and amplify it in a loop. Wave solves this with a **dual engine** design—two completely independent `AVAudioEngine` instances with no shared nodes.



Isolation guarantee. The two engines share no audio nodes, no buffers, and no graph connections. The only coupling is the scalar boost parameter passed from the policy to the masking generator wrapper via `OSAllocatedUnfairLock`. Session configuration (`AVAudioSession`) is managed externally, above both engines.

Figure 12. Dual audio engine design. The playback and microphone paths are fully independent `AVAudioEngine` instances. The only coupling is a scalar boost parameter.

: Dual Engine Isolation

The microphone input and audio output must never share an `AVAudioEngine` instance. Two independent engines eliminate any possibility of feedback loops and allow independent lifecycle management (the mic engine can be started/stopped without affecting playback).

9.2 Spectral Analysis Pipeline

The `MicrophoneMonitor` installs a tap on the input node of its dedicated engine, capturing 4096-frame buffers (≈ 93 ms at 44.1 kHz). The tap callback does *nothing* except store the latest buffer reference—all processing is deferred to a 0.5-second timer.

On each timer tick, the `NoiseAnalyzer` processes the most recent buffer:

1. Extract mono channel 0 from the `AVAudioPCMBuffer`
2. Take min(frameCount, 1024) samples
3. Apply a Hann window via `vDSP_hann_window` [14]
4. Compute Root Mean Square (RMS) energy via `vDSP_rmsqv`
5. Convert to dB: $L = 20 \cdot \log_{10}(\text{RMS} + 10^{-10})$
6. Clamp to $[-96, 0]$ dB

Insight: *Analysing at 2 Hz rather than on every mic callback saves 5–7% battery. Environmental noise evolves on the timescale of seconds (doors opening, HVAC cycling, snoring), not milliseconds. Processing every ~ 93 ms buffer would waste CPU on redundant analysis.*

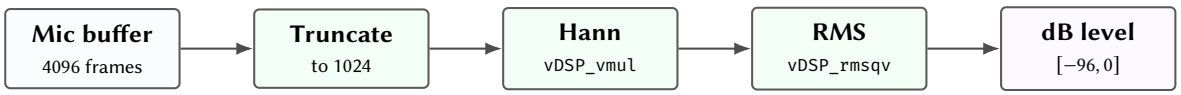


Figure 13. Noise analysis pipeline. Hann windowing reduces spectral leakage for potential future FFT extension; currently, only broadband RMS energy is extracted.

The Hann window [19] is applied even though the current implementation extracts only broadband energy. This serves two purposes: (1) it reduces the contribution of transient edge effects in the buffer, and (2) it keeps the infrastructure ready for a future per-band spectral analysis extension.

9.3 Control Law

The `AdaptiveMaskingPolicy` converts the dB measurement into a volume boost factor through a multi-stage control law.

9.3.1 EMA Smoothing

Raw dB measurements are noisy—a single cough or rustle can produce a transient spike. An Exponential Moving Average (EMA) filter [20] smooths the input:

$$L_{\text{smooth}}[k] = \alpha \cdot L[k] + (1 - \alpha) \cdot L_{\text{smooth}}[k-1] \tag{21}$$

with $\alpha = 0.25$ at a 0.5 s sampling interval, giving a half-life of:

$$t_{1/2} = \frac{-T_s \cdot \ln 2}{\ln(1-\alpha)} = \frac{-0.5 \cdot 0.693}{\ln 0.75} \approx 1.2 \text{ s} \tag{22}$$

This means a sudden transient (e.g., a door slam) is attenuated by 50% within 1.2 seconds and by 87.5% within 3.6 seconds, preventing the system from reacting to brief events that don't require sustained volume adjustment.

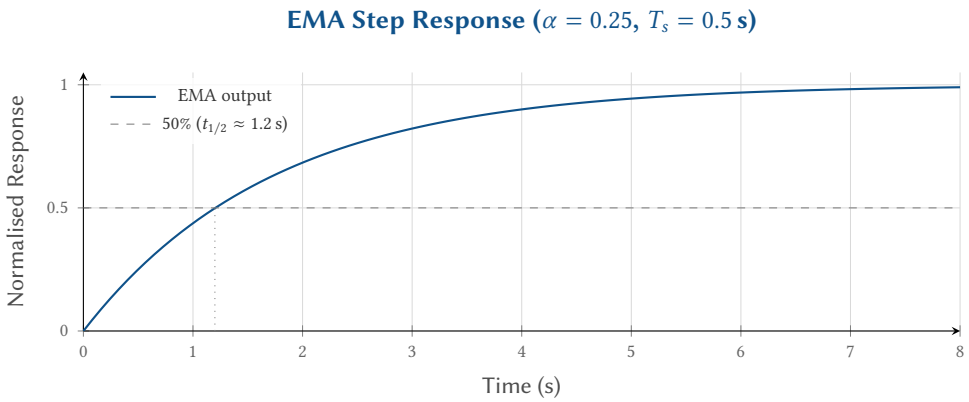


Figure 14. EMA step response. The filter reaches 50% of a sustained step change in ≈ 1.2 s, providing effective transient rejection while tracking sustained ambient changes.

9.3.2 Boost Mapping with Tanh Saturation

The smoothed dB level is mapped to a boost factor using a dead band, linear normalisation, and tanh saturation:

$$\delta = L_{\text{smooth}} - L_{\text{thresh}} \tag{23}$$

$$n = \max\big(0,\ \delta/30\big) \tag{24}$$

$$b = 1.0 + 0.5 \cdot \tanh(2n) \tag{25}$$

where $L_{\text{thresh}} = -40$ dB and a ± 2 dB dead band suppresses micro-adjustments below the threshold.

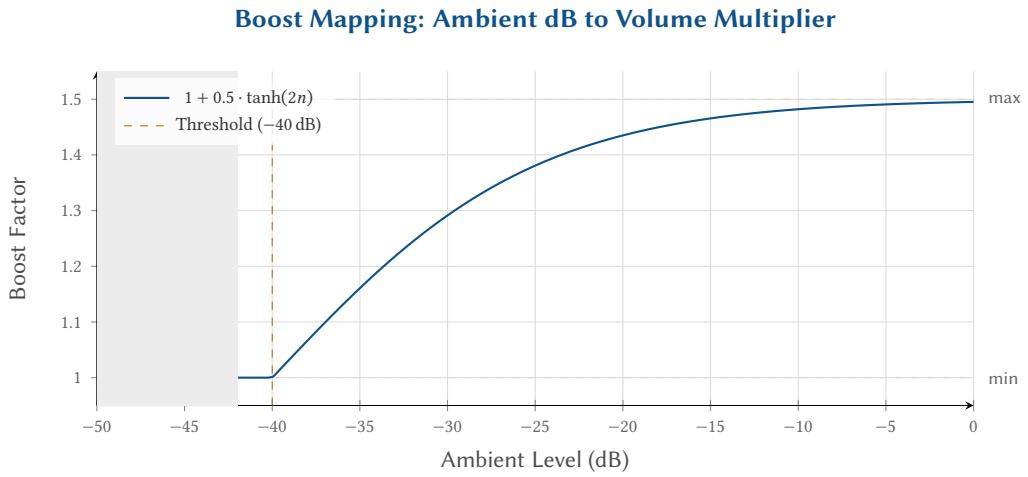


Figure 15. Boost mapping function. The tanh curve provides soft saturation: aggressive response near threshold (where small ambient changes matter most) with gentle limiting as the boost approaches the 1.5× ceiling.

The tanh function is ideal for this application because it provides:

- **Monotonicity**: louder ambient → higher boost, always
- **Soft clipping**: smooth approach to the 1.5× ceiling rather than a hard cutoff
- **Sensitivity near threshold**: the steepest part of the curve is near $n = 0$ (just above threshold), where small ambient changes have the most perceptual impact

Table 6. Adaptive masking policy parameters.

Parameter	Value	Notes
EMA α	0.25	$t_{1/2} \approx 1.2$ s at 2 Hz
Dead band	±2.0 dB	Suppresses micro-adjustments
Trigger	−40 dB	Approx. conversation level
Max boost	1.5×	Hard ceiling
Min boost	1.0×	No volume reduction
Rate limit	0.005/cycle	Full sweep in ≈ 50 s

9.3.3 Rate Limiting

An additional rate limiter constrains how fast the boost can change per analysis cycle:

$$b'[k] = \begin{cases} b_{\text{target}} & \text{if } |b_{\text{target}} - b[k-1]| \leq 0.005 \\ b[k-1] + 0.005 \cdot \text{sign}(b_{\text{target}} - b[k-1]) & \text{otherwise} \end{cases} \tag{26}$$

At 0.005 boost units per 0.5 s cycle, a full swing from 1.0 to 1.5 takes approximately 50 seconds. This prevents perceptible volume jumps even if the EMA-smoothed level changes abruptly.

9.4 Render-Thread Application

The `AdaptiveMaskingGenerator` applies the boost in the audio render callback using an exponential ramp for imperceptible transitions:

$$b_{\text{current}} \mathrel{+}= (b_{\text{target}} - b_{\text{current}}) \cdot r \tag{27}$$

where $r = 256/(30 \cdot 24000) \approx 0.000356$. This gives an effective time constant of ≈ 30 s for a full swing—far below the threshold of perceptibility for gradual volume changes [16].

Parameters are read from `OSAllocatedUnfairLock<MaskingParameters>` once per 256-sample buffer (≈ 10.7 ms at 24 kHz), not per sample.

Part III

System Architecture

10 Playback State Machine

The `PlaybackCoordinator` manages a four-state machine with an overlaid recovery subsystem for Bluetooth resilience.

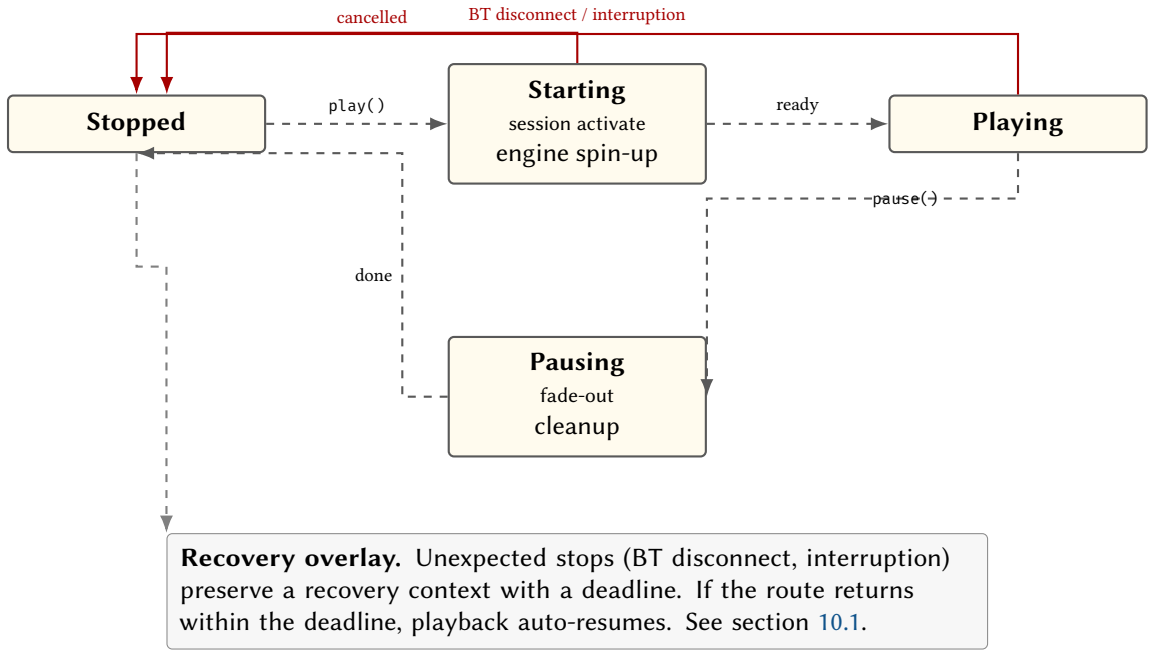


Figure 16. Playback state machine. Normal transitions follow the horizontal path; exception paths (red) bypass the fade-out and transition directly to Stopped with recovery context preserved.

10.1 Recovery Subsystem

When playback is interrupted unexpectedly, the coordinator enters a recovery state rather than simply stopping. Each recovery context carries a deadline, after which the system gives up:

Table 7. Recovery deadlines by interruption type.

Event	Deadline	Rationale
Audio interruption	30 s	Phone call, Siri, etc.
BT disconnect	1.5 s	Headphone out of range / sleep shift
Output stability	250 ms	Debounce before auto-resume

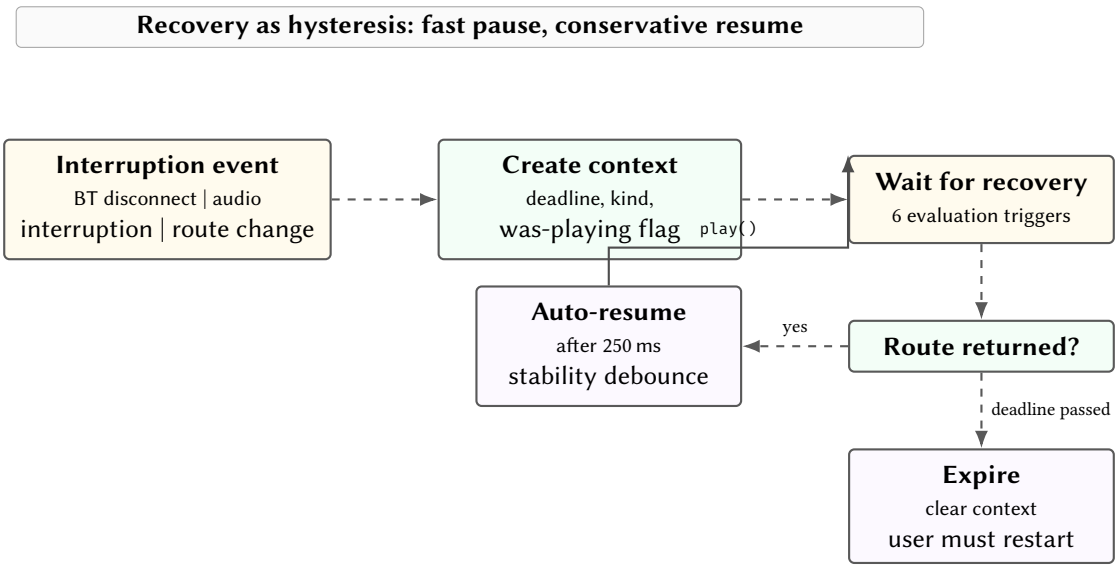


Figure 17. Recovery subsystem flow. The system transitions quickly to "safe" (stopped) on route loss but requires route stability confirmation before auto-resuming.

Recovery evaluation is triggered by six event sources: `interruptionBegan`, `interruptionEnded`, `outputConnectivityChanged`, `appBecameActive`, `userTappedResumeNow`, and `recoveryExpiry`.

10.2 Auto-Resume Strategy

Insight: *The hysteresis pattern—fast to pause, slow to resume—is critical for preventing two failure modes: (1) speaker leakage when headphones disconnect temporarily (e.g., user rolls in sleep), and (2) oscillation during flaky Bluetooth where the route connects and disconnects rapidly.*

Three user-selectable strategies govern whether the system auto-resumes after an interruption or requires manual restart:

Table 8. Auto-resume strategies and their evaluation logic.

Strategy	BT Reconnect	Interruption	Use Case
Smart (default)	Auto	Only if system `shouldResume`	Catches brief Siri queries; ignores phone calls
Always	Auto	Auto	Maximum continuity; may resume after long calls
Never	Manual	Manual	Full user control; safest for shared environments

The evaluation function considers four inputs: the configured strategy, the interruption kind (audio interruption vs. output disconnected), the system's `shouldResume` flag from `AVAudioSession.InterruptionOptions`, and the elapsed time since the interruption began.

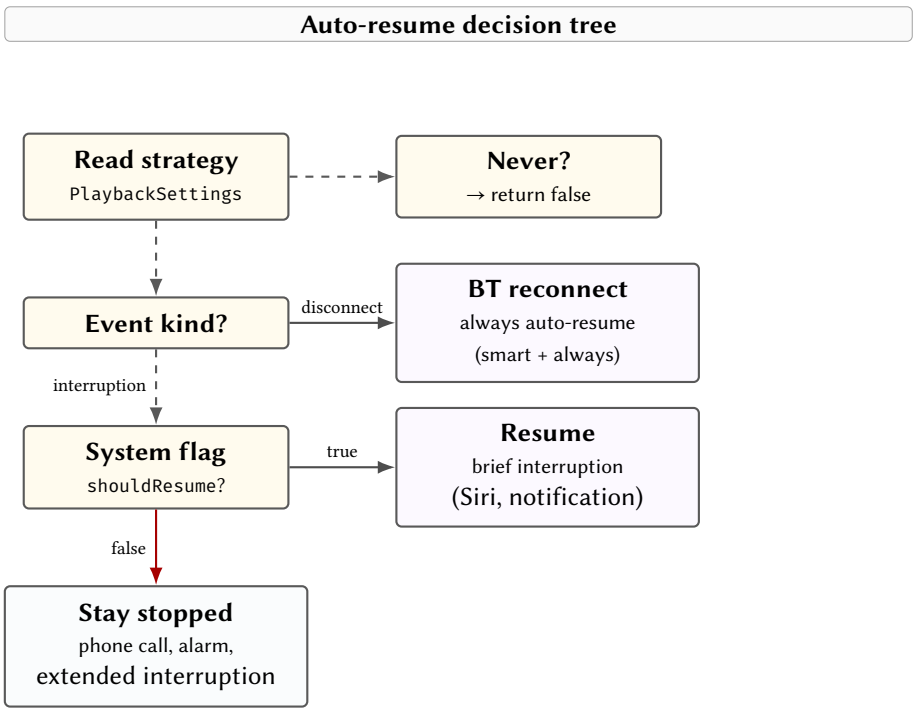


Figure 18. Auto-resume decision tree. Bluetooth reconnects always resume (unless `never`); audio interruptions consult the system's `shouldResume` flag.

11 Real-Time Thread Safety

Insight: *The "smart" default strikes a balance: brief system interruptions (Siri, notification sounds) auto-resume seamlessly, while extended interruptions (phone calls, alarms) leave the user in control. The 30-second recovery deadline ensures that stale contexts are cleaned up promptly.*

The audio render callback runs on a real-time thread managed by Core Audio. Apple's real-time audio guidelines [21] impose strict constraints: no memory allocation, no Objective-C message dispatch, no locks held for extended periods, and deterministic execution time within the render deadline (≈ 10.7 ms at 24 kHz for a 256-sample buffer).

11.1 Lock Protocol

Wave uses `OSAllocatedUnfairLock` (introduced in iOS 16) with a **full value-copy** semantic: the audio thread acquires the lock only to copy a value-type struct in or out, then releases immediately. All frame processing occurs outside the lock.

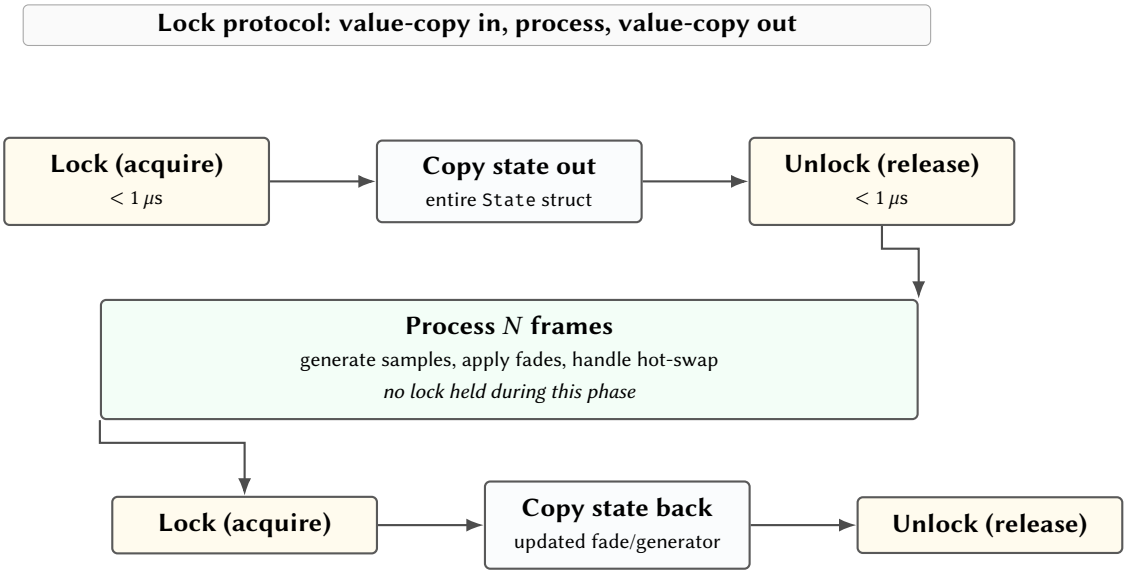


Figure 19. Lock protocol for the render callback. The lock is held only for struct copies ($< 1\ \mu$s); all audio processing runs lock-free.

: Value-Copy Lock Protocol

The audio thread acquires the lock only to copy a `Sendable` value-type struct. All per-frame processing runs outside the lock. Lock hold times remain below 1 μs regardless of buffer size, eliminating priority inversion risk.

Two lock instances exist:

1. **WaveAudioEngine** — A `State` struct containing the current generator, pending hot-swap generator, fade phase/gain, and user volume. Copied in and out on every render callback.
2. **AdaptiveMaskingGenerator** — A `MaskingParameters` struct containing the current boost target. Read once per 256-sample buffer refill.

11.2 Generator Hot-Swap

When the user changes noise type or a parameter triggers chain rebuild, the new generator is written as `pendingGenerator` into the locked state. The render callback detects this and performs a cross-fade:

1. **Fade out** current generator over 480 samples (20 ms at 24 kHz), decrementing `fadeGain` by 1/480 per sample
2. At zero crossing (`fadeGain` = 0): swap `pendingGenerator` into `generator`
3. **Fade in** new generator over 480 samples

The swap occurs at the exact sample where gain crosses zero, guaranteeing zero-energy transition and eliminating audible clicks. The same mechanism is used for startup (fade from silence) and shutdown (`stopWithFade`).

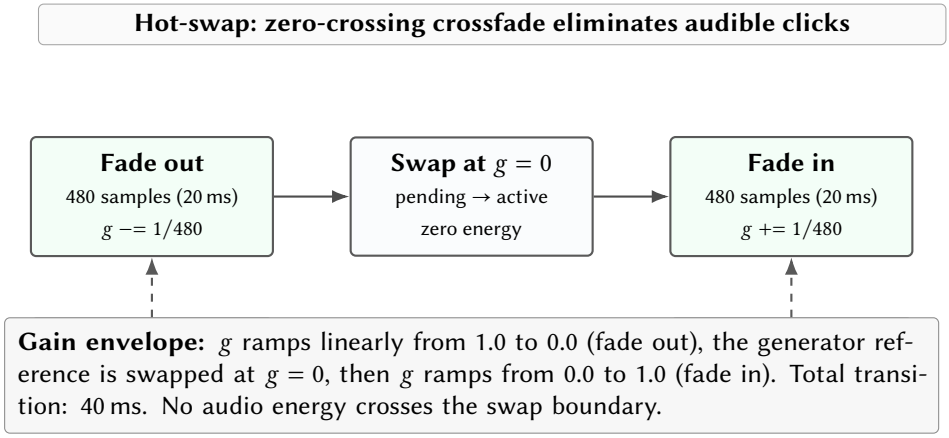


Figure 20. Generator hot-swap via zero-crossing crossfade. The old generator fades to silence, the reference is swapped at zero energy, and the new generator fades in. Total transition: 40 ms (960 samples).

11.3 Startup and Shutdown Fade

A separate software fade envelope operates at the `AudioEngine` level (above `WaveAudioEngine`) to eliminate clicks during session start and stop. Unlike the sample-accurate hot-swap crossfade, this fade uses a Swift `async` loop:

Table 9. Two-layer fade architecture.

Layer	Duration	Steps	Mechanism
AudioEngine (start/stop)	100 ms	10	Swift `Task.sleep`, linear volume ramp at 10 ms intervals
WaveAudioEngine (hot-swap)	20 ms	480	Sample-accurate linear gain in render callback

The startup fade ramps volume from 0 to the target over 100 ms (10 steps of 10 ms). The shutdown fade reverses this, then calls `engine.stop()`. This coarser fade is acceptable for start/stop because the user initiates these transitions consciously; the finer hot-swap fade must be imperceptible during a continuous listening session.

12 Bluetooth Routing

Audio routing is critical for a sleep application: accidental speaker playback can wake a sleeping partner. The `AudioSessionManager` enforces Bluetooth-only output.

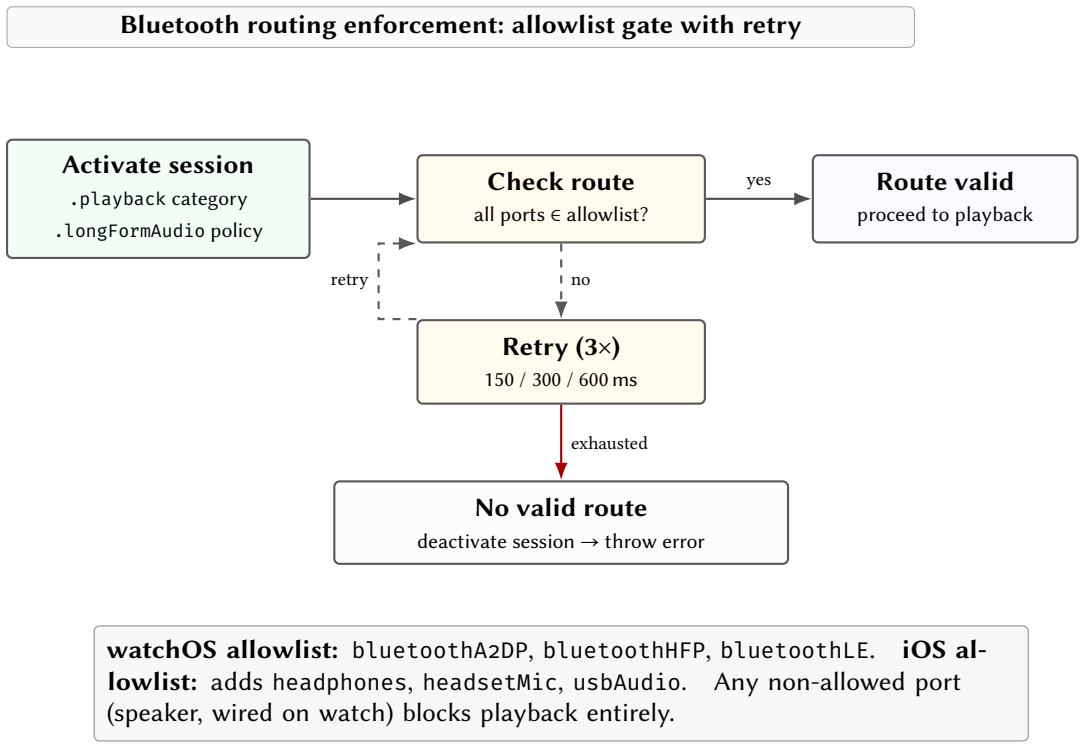


Figure 21. Bluetooth routing enforcement. After session activation, all output ports are validated against a platform-specific allowlist. Up to three retries with exponential backoff handle watchOS routing lag.

12.1 Route Policy

The session is configured with:

```
session.setCategory(.playback, mode: .default,
                    policy: .longFormAudio, options: [])
```

The `.longFormAudio` policy signals to watchOS that this is sustained audio (like a podcast), which affects Bluetooth routing priority and allows the session to continue with the screen off.

All output ports are validated against an allowlist:

- **watchOS**: Bluetooth A2DP, HFP, and BLE only
- **iOS**: additionally allows wired headphones, headset mic, and USB audio

If any non-allowed port is detected, the session is deactivated and playback is blocked with a `noBluetoothHeadphones` error.

12.2 Route Activation Retry

watchOS has a known routing lag where the Bluetooth route may not be immediately active after session activation. The manager retries with exponential backoff:

Table 10. Route activation retry schedule.

Attempt	Delay	Cumulative
1	150 ms	150 ms
2	300 ms	450 ms
3	600 ms	1050 ms

If no valid route is active after all retries, the session is immediately deactivated and an error is thrown.

13 Audio Session and Interruption Handling

The `AudioSessionManager` centralises all interaction with `AVAudioSession` [22], handling configuration, route observation, and interruption lifecycle. On macOS the entire class compiles as a no-op stub via `#if os(macOS)`.

13.1 Session Configuration

The session is configured for sustained background audio:

```
try session.setCategory(
    .playback, mode: .default,
    policy: .longFormAudio, options: [])
```

The `.longFormAudio` routing policy signals to watchOS that this is sustained audio (analogous to a podcast), which:

- Elevates Bluetooth routing priority over system sounds
- Permits audio to continue with the screen off
- Prevents the system from pausing audio on wrist-down

13.2 Interruption Lifecycle

The manager observes `AVAudioSession.interruptionNotification` and translates raw system events into a structured notification:

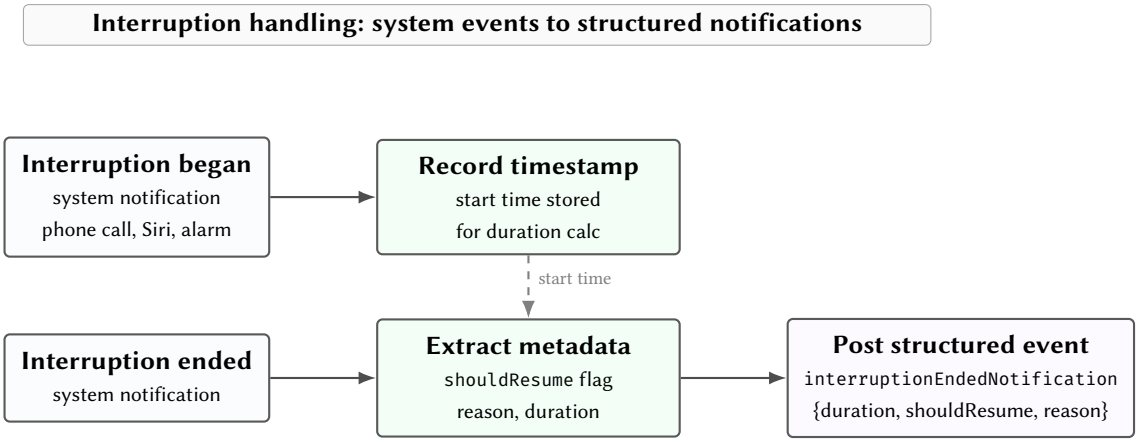


Figure 22. Interruption lifecycle. The manager translates raw system notifications into structured events carrying duration, the `shouldResume` flag, and the interruption reason.

13.3 Route Change Observation

The manager also observes `AVAudioSession.routeChangeNotification` and dispatches on the change reason:

- **`.oldDeviceUnavailable`**: Headphone disconnection detected. Sets `isBluetoothConnected = false` and triggers recovery evaluation.
- **`.newDeviceAvailable`**: New audio device connected. Verifies the route against the allowlist and updates `isBluetoothConnected`.
- **All other reasons**: Re-evaluates the routing policy to confirm the current route is still in the allowlist.

The manager also exposes a `BeatsOutputCapability` struct indicating whether the current route supports binaural beats:

```
public struct BeatsOutputCapability {
    public let isBluetoothActive: Bool
    public let outputChannels: Int
    public let isBeatsCapable: Bool  // BT + stereo
}
```

Binaural beats require true stereo separation; mono Bluetooth output (some Bluetooth Low Energy (BLE) accessories) collapses the dichotic image and is rejected.

: Centralised Session Management

All `AVAudioSession` interaction flows through a single `AudioSessionManager`. Neither the playback engine nor the microphone engine configures the session directly. This prevents conflicting category/policy settings and ensures route observation fires exactly once per change event.

14 FlowState: macOS Menu Bar App

FlowState is a lightweight macOS menu bar application that provides the same synthesis engine for focus work. It uses the shared `WaveAudioKit` package directly, without the watchOS-specific layers (no microphone monitoring, no Bluetooth enforcement, no session management).

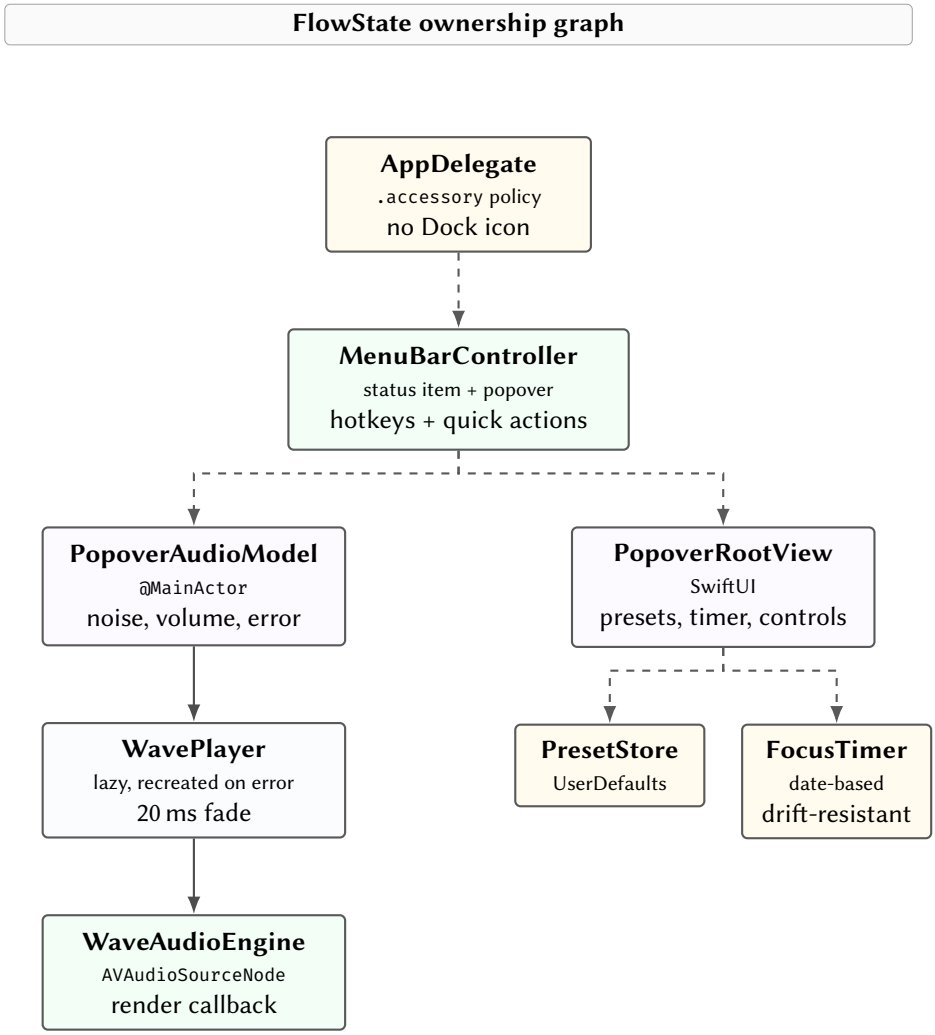


Figure 23. FlowState ownership graph. The `MenuBarController` owns both the audio model and the SwiftUI popover. The `WavePlayer` is lazily created and destroyed on error for cold-start safety.

14.1 Global Hotkeys

FlowState registers three global keyboard shortcuts using the Carbon `RegisterEventHotKey` API (the only macOS API that supports truly global hotkeys that work regardless of which app is focused):

Table 11. FlowState global hotkeys.

Shortcut	Action
`Ctrl+Opt+Cmd+P`	Toggle play/pause
`Ctrl+Opt+Cmd+O`	Toggle popover
`Ctrl+Opt+Cmd+N`	Cycle noise type

Hotkey callbacks dispatch to `@MainActor` via `Task { @MainActor in }` to ensure thread safety with the SwiftUI state model.

14.2 Focus Timer

The focus timer uses a date-based countdown (storing an `endDate` and computing remaining time as $t_{\text{end}} - t_{\text{now}}$) rather than decrementing a counter on each tick. This is drift-resistant: even if timer callbacks are delayed by system load, the displayed time remains accurate.

When the timer completes, audio is auto-paused and a brief "Done" confirmation is displayed for 3 seconds. Starting the timer also auto-starts audio if not already playing.

14.3 Presets

Named audio configurations are persisted via `UserDefaults` (JSON-encoded). Each preset captures a noise type and volume level. Two defaults ship with the app: "Focus" (brown, 0.65) and "Soft" (blend, 0.55).

15 watchOS Interaction: Digital Crown and Hand Gestures

Sleep applications face a unique interaction constraint: the user may be in bed, in the dark, with limited dexterity. Wave provides two screen-free input mechanisms that allow control without visual attention.

15.1 Digital Crown Volume Control

The Digital Crown is bound to volume via SwiftUI's `.digitalCrownRotation` modifier with an **inverted** mapping: rotating the crown forward (away from the user) decreases volume. This matches the watchOS convention where "forward" scrolls down, and intuitively maps to "quieter" in a sleep context.

Table 12. Digital Crown configuration parameters.

Parameter	Value
Range	0.0 – 1.0
Step size	0.01 (1% per detent)
Sensitivity	`.medium`
Continuous	`false` (stops at limits)
Haptic feedback	`true` (detent clicks)
Default volume	0.35

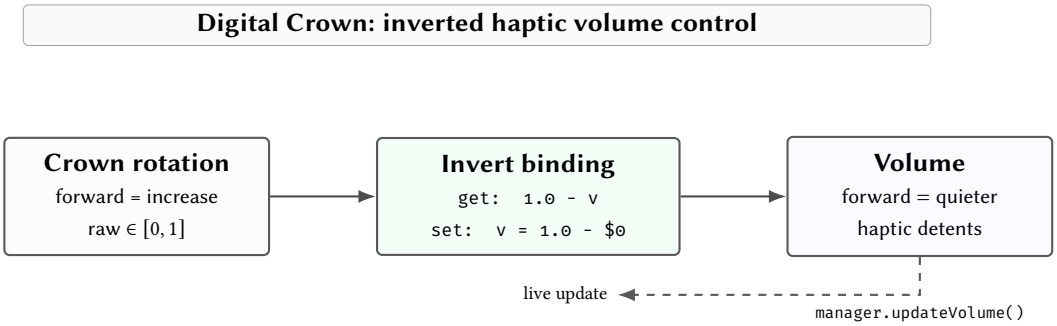


Figure 24. Digital Crown volume binding. The inversion ensures that rotating forward (the natural "scrolling" direction) reduces volume, matching sleep-context intuition.

The view declares `.focusable(true)` to receive crown events, and syncs the crown position to the current volume on appear.

15.2 Double-Pinch Gesture

On Apple Watch Ultra and Series 9+, the `.handGestureShortcut(.primaryAction)` modifier binds the double-pinch (thumb-to-index-finger) gesture to play/pause toggle. This allows the user to control playback without raising their wrist or touching the screen—critical for sleep use where physical interaction can cause arousal.

16 Session Telemetry and Morning Summary

Insight: *The combination of crown volume and double-pinch play/pause provides complete audio control without ever looking at the screen. A sleeping user can adjust volume by feel (haptic detents provide positional feedback) and toggle playback with a hand gesture.*

Each playback session is tracked as a `SessionState` struct, persisted to `UserDefaults` on every mutation for crash safety. The telemetry subsystem serves two purposes: post-session user feedback (the Morning Summary) and diagnostic data for battery optimisation.

16.1 Session State Model

Table 13. SessionState fields and their sources.

Field	Type	Source
`id`	UUID	Generated at session start
`startTime`	Date	Captured at `play()`
`endTime`	Date?	Set at `pause()` or expiry
`startBatteryLevel`	Float	`BatteryMonitor` snapshot
`endBatteryLevel`	Float?	Final snapshot at stop
`interruptionCount`	Int	Incremented per interruption event
`failures`	[FailureEvent]	Recovery attempts and outcomes
`wasCompletedNormally`	Bool	True if user-initiated stop
`adaptiveMonitoringSeconds`	Double	Cumulative mic-active time
`adaptationTriggerCount`	Int	Number of boost adjustments

Each `FailureEvent` records a timestamp, reason string, recovery attempt count, and whether recovery succeeded. This enables post-session diagnostics: a session with many failures that recovered indicates flaky Bluetooth; many unrecovered failures suggest incompatible headphones.

16.2 Crash-Safe Persistence

The `SessionTracker` writes the active session to `UserDefaults` after every field mutation, calling `synchronize()` to force an immediate write. If the app crashes or is terminated by the system, `recoverSession()` on next launch reads the incomplete session back and either resumes it (if less than 1 hour has elapsed) or appends it to history as an abnormal completion.

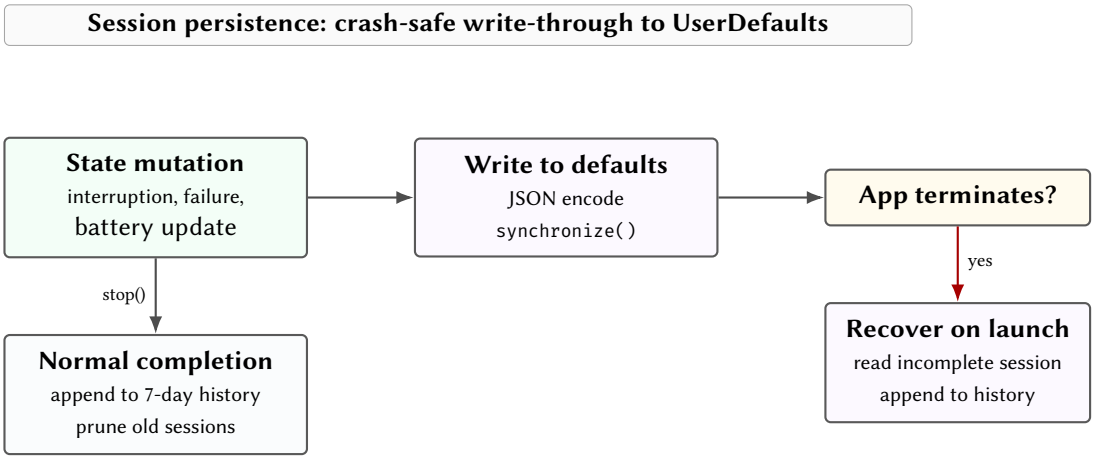


Figure 25. Session persistence with crash recovery. Every mutation is written through to `UserDefaults` immediately; incomplete sessions are recovered on next launch.

16.3 Morning Summary

Completed sessions are stored in a 7-day rolling history (pruned on every write). When the user opens the app, a Morning Summary is displayed if the most recent session meets three criteria:

1. Duration $>$ 4 hours (a real sleep session, not a quick test)
2. Ended more than 1 hour ago (not currently in use)
3. Ended less than 24 hours ago (still relevant)

The summary presents session duration, battery drain, interruption count, and recovery success rate—giving the user a quick digest of how their sleep audio performed overnight.

17 Battery Monitoring

Battery life is the primary constraint for an all-night watchOS application. Min et al. [23] characterised smartwatch battery utilisation in the wild, finding that background audio and sensor polling are among the highest-drain activities. The `BatteryMonitor` provides real-time drain rate estimation and remaining-time projection, feeding both the UI and the session telemetry system.

17.1 Drain Rate Estimation

The monitor captures the battery level at session start and polls `WKInterfaceDevice.batteryLevel` every 5 minutes via a Combine timer. The drain rate in percent per hour is:

$$r = \frac{(L_{\text{start}} - L_{\text{current}}) \times 100}{(t_{\text{now}} - t_{\text{start}})/3600} \tag{28}$$

Remaining hours are projected as:

$$t_{\text{remaining}} = \frac{L_{\text{current}} \times 100}{r} \tag{29}$$

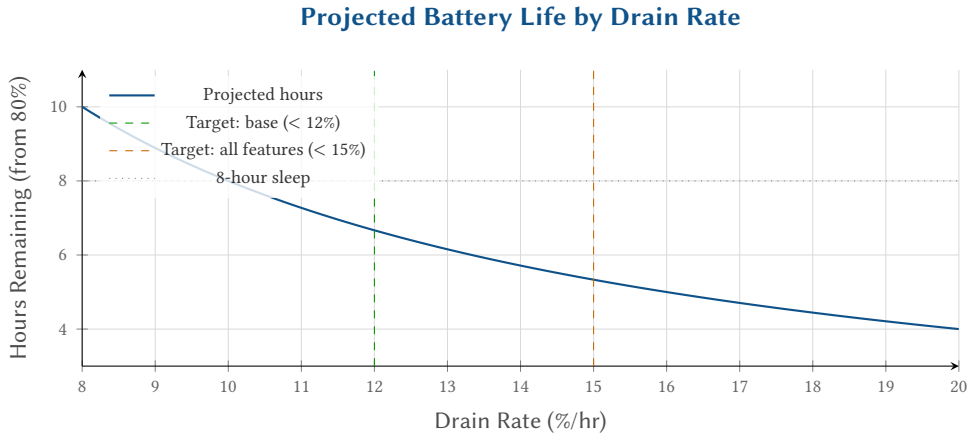


Figure 26. Projected remaining battery life from 80% charge at various drain rates. At the 12%/hr target (base playback), 6.7 hours remain; at 15%/hr (all features), 5.3 hours. An 80%+ starting charge supports a full 8-hour session at base drain rates.

17.2 Thresholds and Alerts

A critical threshold fires at 10% battery remaining, setting a published `isCritical` flag that the UI consumes to display a warning. The drain rate is colour-coded in the Battery Metrics view:

Table 14. Battery drain rate colour thresholds.

Colour	Rate (%/hr)	Interpretation
Green	< 12	On target for 8-hour session
Orange	12–15	Acceptable with features enabled
Red	> 15	May not last full night

Battery data feeds directly into `SessionState`: the start and end levels are captured as snapshots, and the computed drain rate appears in the Morning Summary.

Insight: *The 5-minute polling interval is a deliberate trade-off: more frequent polling would provide smoother rate estimates but adds timer wake-ups that themselves consume battery. At 5 minutes, the drain rate stabilises within the first 15 minutes (3 data points) and the timer contribution to battery drain is negligible.*

18 Multi-Platform Architecture

Wave is structured as a set of shared Swift packages consumed by platform-specific thin app shells. This architecture allows the same `PlaybackCoordinator` to drive both watchOS (StillState—sleep) and macOS (FlowState—focus), with only platform integration points varying.

18.1 Package Structure

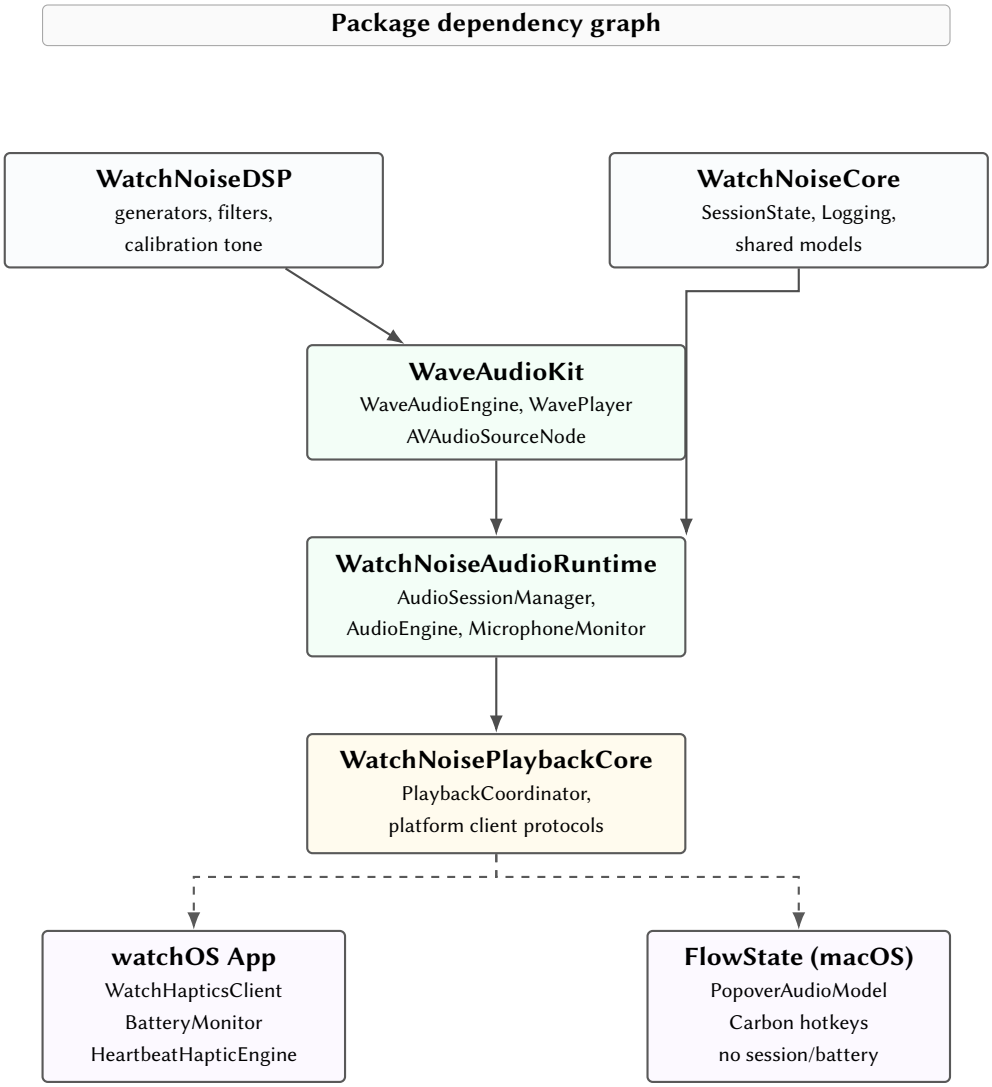


Figure 27. Package dependency graph. Shared packages form a vertical stack; platform-specific app shells inject concrete client implementations at the bottom.

18.2 Platform Client Protocols

The `PlaybackCoordinator` depends on three protocols, each with platform-specific implementations:

Table 15. Platform client protocols and their implementations.

Protocol	watchOS	iOS / macOS
`HapticsClient`	`WKInterfaceDevice.play()` for transient events; `HeartbeatHapticEngine` for rhythmic pulses	iOS: `UIImpactFeedbackGenerator`; heartbeat is no-op (no wrist). macOS: all no-ops
`BatteryClient`	Full `BatteryMonitor`: 5-min polling, drain rate, `isCritical`, remaining projection	iOS: thin `UIDevice.batteryLevel` wrapper. macOS: not applicable
`AppLifecycleClient`	`WKExtension` resign/active notifications	iOS: `UIApplication` resign/active. macOS: not used

All protocols are marked `@MainActor` to ensure thread safety with SwiftUI state. The coordinator itself is entirely platform-agnostic: it receives the three clients at init time and never imports `WatchKit`, `UIKit`, or `AppKit`.

: Protocol-Injected Platform Abstraction

Platform-specific behaviour is expressed as protocol conformances injected at app startup. The shared `PlaybackCoordinator` never conditionally compiles for a specific platform. Adding a new platform (e.g., visionOS) requires only implementing the three client protocols.

Insight: *This architecture emerged from practical necessity: the watchOS and macOS targets share 85%+ of their audio logic. Without protocol injection, the coordinator would be riddled with `#if os(watchOS)` conditionals that make testing impossible. The protocol approach allows the coordinator to be tested with mock clients in a plain Swift package target.*

19 Key Metrics

Table 16. System parameters and performance characteristics.

Parameter	Value	Notes
Playback sample rate	24 kHz	Sufficient for sleep noise; halves CPU vs 48 kHz
Analysis window	1024 pt	Hann window, vDSP accelerated
Analysis rate	2 Hz	0.5 s timer; saves 5–7% battery
Mic tap buffer	4096 fr	$\approx$ 93 ms at 44.1 kHz
DSP block size	256 smp	$\approx$ 10.7 ms at 24 kHz
Fade duration	20 ms	480 samples; click-free transitions
Max adaptive boost	1.5×	Hard ceiling (soft tanh saturation)
Boost ramp τ	$\approx$ 30 s	Exponential, imperceptible
Heartbeat depth	5–8%	Below JND for amplitude modulation
Calibration boost	+4 dB	Peaking EQ, $Q = 1.0$, $\approx$ 1 octave BW
Blend LFO period	20 s	±10% white/brown swing
Brown b_1 (std)	0.995	$f_c = 20$ Hz; 6 dB/oct rolloff
Phase precision	Float64	Eliminates pitch drift over 8 hr sessions
Lock hold time	$< 1\,\mu$s	Value-copy protocol
Battery (base)	$<$ 12%/hr	Playback only, screen off
Battery (all features)	$<$ 15%/hr	Calibration + heartbeat + adaptive
Codebase	$\approx$ 7860 LOC	78 Swift files across 4 packages

20 Development Infrastructure

20.1 Structured Logging

All runtime subsystems log via Apple's unified logging framework [24], using three category-specific loggers under a shared subsystem:

Table 17. OSLog subsystem categories.

Logger	Category	Scope
`Logger.audio`	`audio`	Engine lifecycle, playback events, generator swaps
`Logger.session`	`session`	Session activation, route changes, interruptions
`Logger.battery`	`battery`	Drain rate, critical alerts, monitoring lifecycle

All three loggers share the subsystem `com.watchnoise.app`, enabling Instruments and Console filtering by subsystem or category. Privacy-sensitive data (battery levels, session durations) uses the `.private` redaction level in release builds.

20.2 Simulator Guards

The audio stack cannot run in the watchOS Simulator (`AVAudioEngine` triggers a `libpthread` trap). Wave handles this with `#if targetEnvironment(simulator)` guards across six files:

- **AudioEngine / WaveAudioEngine**: all `start()`, `stop()`, `switchGenerator()`, and `setVolume()` calls become no-ops. State machines still transition normally.
- **AudioSessionManager**: route capability checks return permissive values (`isBluetoothActive = true`, stereo channels), allowing UI flows to proceed without hardware.
- **BatteryMonitor**: skips `WKInterfaceDevice.batteryLevel` reads; returns a fixed level that decrements by 1% per tick for visual testing.

- **CalibrationEngine**: sets `isPlaying` state without starting the tone engine.
- **ContentView**: renders a "Simulator: audio disabled" banner to make the no-op state visible.

20.3 TestFlight Build Channel

A `BuildChannel` enum detects the distribution channel at runtime by inspecting the App Store receipt URL:

```
static var isTestFlight: Bool {
    #if DEBUG
    return false
    #else
    return Bundle.main.appStoreReceiptURL?
        .lastPathComponent == "sandboxReceipt"
    #endif
}
```

Insight: *The simulator guard strategy preserves all state machine transitions and UI flows while suppressing only hardware-dependent calls. This allows the full app lifecycle—including recovery, hot-swap, and calibration—to be exercised in the Simulator for rapid iteration.*

In TestFlight builds, a hidden gesture on the About screen triggers a controlled crash via `fatalError` with a known signature string to verify that crash symbolication is working in App Store Connect. This mechanism is gated behind `BuildChannel.allowsControlledCrash` and requires a confirmation alert before firing.

20.4 Debug Route Override

In `DEBUG` builds, an `AudioRouteDebugView` provides buttons to override the reported output port types without physical hardware:

- Simulate Bluetooth Advanced Audio Distribution Profile (A2DP) connected
- Simulate built-in speaker (non-Bluetooth)
- Simulate no outputs (empty route)
- Clear override (restore real hardware)
- "Simulate bounce"—scripted disconnect/reconnect with 800 ms gap to exercise recovery logic

Each override sets `AudioSessionManager.debugOverrideOutputPortTypes` and triggers an immediate route re-evaluation, allowing the full recovery and Bluetooth routing enforcement flow to be tested on the Simulator.

21 Conclusion and Future Work

Wave demonstrates that procedural audio synthesis is a viable alternative to pre-recorded loops for sleep and focus applications, even on the constrained hardware of Apple Watch. The key architectural contributions are:

1. **Wrapper composition** over monolithic processing: each audio processing layer (bias, heartbeat, adaptive masking) is a transparent decorator around the generator protocol, enabling zero-coupling feature addition and runtime reconfiguration.
2. **Dual-engine isolation** for adaptive masking: separating the microphone and playback paths into independent `AVAudioEngine` instances eliminates feedback loops by construction.
3. **Value-copy lock protocol** for real-time safety: the audio render thread never holds a lock during processing, achieving $< 1\,\mu s$ lock hold times regardless of buffer size.
4. **Protocol-injected platform abstraction**: the shared `PlaybackCoordinator` runs identically on watchOS, iOS, and macOS, with only three protocol conformances varying per platform.

21.1 Planned Extensions

Several extensions are in active planning:

Environment Synthesis (Phase 27) Four procedural environment generators (rain, café, traffic, nature) using filtered pink noise combined with stochastic sample triggering via a Poisson process. These will layer on top of existing noise types via the wrapper composition architecture—not replace them.

iOS Standalone App (Phase 22) A full iOS target consuming the same shared packages, with a headphones-only routing policy (allowing wired and USB audio in addition to Bluetooth). The v1.6 milestone constrains this to local-only operation: no WatchConnectivity, no iCloud sync.

HealthKit Integration Recording sleep session data (duration, interruption count, battery drain) as HealthKit category samples, enabling correlation with Apple Health sleep analysis. Currently deferred pending privacy review.

Watch Complications A complication showing current playback state and battery projection on the watch face, enabling one-tap session start without launching the full app.

WatchConnectivity Phone-to-watch settings sync for users who prefer configuring calibration and presets on the larger iPhone screen. Explicitly deferred past v1.6.

> **Design Decision 2: Extensibility via Composition**
>
> Every planned feature maps cleanly onto the existing architecture. Environment synthesis adds a new wrapper. iOS support injects new platform clients. HealthKit writes to a new data sink. No existing subsystem requires modification—only addition. This is the primary validation of the wrapper composition and protocol injection patterns.

References

[1] M. L. Stanchina, M. Abu-Hijleh, B. K. Chaudhry, C. C. Carlisle, and R. P. Millman, "The influence of white noise on sleep in subjects exposed to ICU noise," *Sleep Medicine*, vol. 6, no. 5, pp. 423–428, 2005. DOI: 10.1016/j.sleep.2004.12.004

[2] L. Messineo, L. Taranto-Montemurro, S. A. Sands, M. D. Oliveira Marques, A. Azabarzin, and D. A. Wellman, "Broadband sound administration improves sleep onset latency in healthy subjects in a model of transient insomnia," *Frontiers in Neurology*, vol. 8, p. 718, 2017. DOI: 10.3389/fneur.2017.00718

[3] A. S. Bregman, *Auditory Scene Analysis: The Perceptual Organization of Sound*. MIT Press, 1990, ISBN: 978-0262521956.

[4] A. Papoulis and S. U. Pillai, *Probability, Random Variables, and Stochastic Processes*, 4th. McGraw-Hill, 2002, ISBN: 978-0073660110.

[5] C. W. Gardiner, *Stochastic Methods: A Handbook for the Natural and Social Sciences*, 4th. Springer, 2009, ISBN: 978-3540707127.

[6] J. O. Smith, *Introduction to Digital Filters with Audio Applications*. W3K Publishing, 2007. [Online]. Available: https://ccrma.stanford.edu/~jos/filters/

[7] H. W. Dove, "Über die Combinationstöne," *Repertorium der Physik*, vol. 3, pp. 494–500, 1841.

[8] G. Oster, "Auditory beats in the brain," *Scientific American*, vol. 229, no. 4, pp. 94–102, 1973.

[9] J. C. R. Licklider, J. C. Webster, and J. M. Hedlun, "On the frequency limits of binaural beats," *The Journal of the Acoustical Society of America*, vol. 22, no. 4, pp. 468–473, 1950. DOI: 10.1121/1.1906629

[10] T. L. Huang and C. Charyton, "A comprehensive review of the psychological effects of brainwave entrainment," *Alternative Therapies in Health and Medicine*, vol. 14, no. 5, pp. 38–50, 2008.

[11] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994, ISBN: 978-0201633610.

[12] B. C. J. Moore, *An Introduction to the Psychology of Hearing*, 6th. Brill, 2012, ISBN: 978-9004252424.

[13] R. Bristow-Johnson, *Audio EQ cookbook*, 2021. [Online]. Available: https://www.w3.org/2011/audio/audio-eq-cookbook.html

[14] Apple, *vDSP: Digital signal processing with Accelerate*, 2024. [Online]. Available: https://developer.apple.com/documentation/accelerate/vdsp

[15] L. Bernardi, C. Porta, and P. Sleight, "Cardiovascular, cerebrovascular, and respiratory changes induced by different types of music in musicians and non-musicians: The importance of silence," *Heart*, vol. 92, no. 4, pp. 445–452, 2006. DOI: 10.1136/hrt.2005.064600

[16] N. F. Viemeister, "Temporal modulation transfer functions based upon modulation thresholds," *The Journal of the Acoustical Society of America*, vol. 66, no. 5, pp. 1364–1380, 1979. DOI: 10.1121/1.383531

[17] R. T. Azevedo, N. Bennett, A. Bilicki, J. Hooper, F. Markopoulou, and M. Tsakiris, "The calming effect of a new wearable device during the anticipation of public speech," *Scientific Reports*, vol. 7, no. 1, p. 2285, 2017. DOI: 10.1038/s41598-017-02274-2

[18] J. Costa, A. T. Adams, M. F. Jung, F. Guimbretiere, and T. Choudhury, "Emotioncheck: Leveraging bodily signals and false feedback to regulate our emotions," in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, 2016, pp. 758–769. DOI: 10.1145/2971648.2971752

[19] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, 1978. DOI: 10.1109/PROC.1978.10837

[20] J. S. Hunter, "The exponentially weighted moving average," *Journal of Quality Technology*, vol. 18, no. 4, pp. 203–210, 1986. DOI: 10.1080/00224065.1986.11979014

[21] Apple, *AVAudioEngine: Audio processing graph*, 2024. [Online]. Available: https://developer.apple.com/documentation/avfaudio/avaudioengine

[22] Apple, *AVAudioSession: Configure your app's audio session*, 2024. [Online]. Available: https://developer.apple.com/documentation/avfaudio/avaudiosession

[23] C. Min et al., "Understanding smartwatch battery utilization in the wild," *Sensors*, vol. 20, no. 13, p. 3616, 2020. DOI: 10.3390/s20133616

[24] Apple, *Logging: Generate log messages from your code*, 2024. [Online]. Available: https://developer.apple.com/documentation/os/logging